IHK

Abschlussprüfung Winter 2026

Fachinformatiker für Anwendungsentwicklung

Dokumentation zur betrieblichen Projektarbeit

Lern-App

Entwicklung einer webbasierten Lern-App zur Prüfungsvorbereitung

Abgabedatum: Kiel, den 30.11.2025

Prüfungsbewerber:

Abdul Aziz Ilhan

Lorenzenweg 15 a

24782 Büdelsdorf

Ausbildungsbetrieb:

GFN GmbH

Hopfenstraße 1B

24124 Kiel

GFN

Inhaltsverzeichnis

1. Einleitung ... 3
2. Ausgangssituation ... 4
3. Zielsetzung ... 4
4. Projektumfeld ... 6
5. Planung ... 6
5.1 Anforderungen ... 6
5.2 Use Cases ... 7
5.3 Mockups ... 9
5.4 Zeitplanung ... 10
6. Entwurf ... 11
6.1 Systemarchitektur ... 11
6.2 Datenmodell ... 12
6.3 Klassendiagramm ... 13
7. Implementierung ... 16
7.1 Backend-Implementierung ... 16
7.1.1 Architektur und Projektstruktur ... 16
7.1.2 CORS-Konfiguration und REST-Schnittstelle ... 16
7.1.3 Verarbeitung und Speicherung der Ergebnisse ... 17
7.2 Datenbank-Implementierung ... 17
7.2.1 Tabellen und Beziehungen ... 17
7.2.2 Initial Befüllung mit Fragen und Antworten ... 18
7.3 Frontend-Implementierung ... 18
7.3.1 Aufbau und Komponenten ... 18
7.3.2 Ablauf der Quiz-Logik ... 18
8 Testphase ... 19
8.1 Vorgehensweise bei den Tests ... 19
8.2 Testfälle ... 20
8.3 Testauswertung und Qualitätssicherung ... 22
8.4 Zusammenfassung ... 22
9 Wirtschaftlichkeitsbetrachtung ... 22
9.1 Ausgangslage ... 22

9.2 Aufwand 23
9.3 Nutzen für den Ausbildungsbetrieb 23
9.4 Wirtschaftliches Fazit 24
10 Fazit und Reflexion 24

Anhang

Anhang A Backend

A1 Backend-Start & Ping-Test 25
A2 Start und Ausführung des Backends 26
A3 CORS-Konfiguration 26
A4 Test der REST-Schnittstellen (Postman) 27
28
A5 Fehleranalyse & Debugging 29
30
31

Anhang B – Datenbank PostgreSQL

B1 Tabellenübersicht 31
32
B2 Tabellenansichten Frage, Antwort, Benutzer Ergebnis 33
34

Anhang C – Frontend React

C1 Initialer Start des Frontends 35
36
37
38
39
C2 npm / npx 40

Anhang D – Funktionstests
D1 Test: Themenauswahl *41*
42
43
D2 Test: Fragen laden *44*
D3 Test: Antworten & Ergebnisanzeige *45*
D5 Test: Quiz-Abschluss & Neustart *46*

Quellverzeichniss 46
47

1. Einleitung

Im Rahmen meiner Ausbildung zum Fachinformatiker für Anwendungsentwicklung realisiere ich das Projekt Entwicklung einer webbasierten Lern-App zur Prüfungsvorbereitung.
Ziel dieses Projekts ist es, eine Anwendung zu entwickeln, die Auszubildenden und weiteren Lernenden ermöglicht, Multiple-Choice-Fragen online zu beantworten und dadurch ihren Wissensstand gezielt zu überprüfen.

Die App soll flexibel und benutzerfreundlich einsetzbar sein. Sie wird als moderne Client-Server-Anwendung umgesetzt und soll eine praxisnahe, erweiterbare Grundlage für die individuelle Prüfungsvorbereitung schaffen.

2. Ausgangssituation

Im Ausbildungsbetrieb existiert bislang keine zentrale Lösung, um Auszubildende oder Mitarbeitende bei der gezielten Prüfungsvorbereitung zu unterstützen. Stattdessen greifen Lernende auf unterschiedliche externe Quellen und Lernmedien zurück. Diese unterscheiden sich jedoch stark in Qualität, Struktur und Umfang, sodass ein kontinuierlicher und systematischer Lernprozess erschwert wird.

Darüber hinaus fehlt eine Möglichkeit, den eigenen Lernfortschritt zu erfassen und gezielt an Schwächen zu arbeiten. Vorhandene Materialien bieten weder eine Fortschrittsanzeige noch eine automatische Auswertung der Antworten. Dadurch wird es für die Lernenden schwierig, ihren Wissensstand objektiv einzuschätzen.

Dieses Defizit macht deutlich, dass ein digitales Werkzeug erforderlich ist, das Fragen strukturiert bereitstellt, Ergebnisse direkt auswertet und unabhängig von Ort und Gerät genutzt werden kann.

3. Zielsetzung

Ziel des Projekts ist die Entwicklung einer webbasierten Lern-App, die Auszubildenden und weiteren Lernenden eine effiziente und benutzerfreundliche Möglichkeit zur Prüfungsvorbereitung bietet.
Die Anwendung konzentriert sich auf eine themenbasierte Lernstruktur. Nach dem Login kann der Benutzer aus verschiedenen Themengebieten der Abschlussprüfung wählen (z. B. Wirtschafts- und Sozialkunde, Programmierung, IT-Sicherheit). Anschließend werden Fragen aus dem gewählten Themengebiet angezeigt. Dadurch können Lernende gezielt Schwerpunkte setzen und sich individuell auf einzelne Themenbereiche vorbereiten.

Im ursprünglichen Projektantrag war zusätzlich ein Zufallsmodus vorgesehen.
Im Verlauf der Entwicklung wurde dieser zugunsten einer klaren Themenauswahl entfernt,
um den Lernprozess strukturierter, fokussierter und nachvollziehbarer zu gestalten.

Die technische Umsetzung erfolgt auf Basis eines React-Frontends, eines Java-Backends mit Spring Boot und einer PostgreSQL-Datenbank. Über eine REST-API kommunizieren die Anwendungsschichten miteinander.

Die Anwendung soll folgende Kernfunktionen bereitstellen:

- **Benutzerverwaltung:** Registrieren und Anmelden von Nutzern, sodass persönliche Lernfortschritte gespeichert werden können.
- **Fragenpool:** Zentrale Sammlung aller Multiple-Choice-Fragen, die in Themengebiete unterteilt sind.
- **Themenauswahl:** Benutzer wählt ein Thema und startet ein themenspezifisches Quiz
- **Antwortauswertung:** Automatische Bewertung der Antworten mit Anzeige von richtig und falsch beantworteten Fragen.
- **Ergebnisübersicht:** Darstellung des erzielten Ergebnisses sowie optionaler Lernfortschritts.
- **Responsives Design:** Nutzung der Anwendung über verschiedene Endgeräte (PC, Tablet, Smartphone).

Zur Realisierung dieser Ziele wird die Lern-App technisch wie folgt umgesetzt:

- Das **Frontend** wird mit React realisiert.
- Das **Backend** basiert auf Java und Spring Boot.
- Die **Datenhaltung** erfolgt in einer PostgreSQL-Datenbank.
- Die **Kommunikation** zwischen Frontend und Backend läuft über eine REST-Schnittstelle.

Durch diese Lösung wird eine strukturierte, nachvollziehbare und praxisnahe Möglichkeit geschaffen, den Lernprozess gezielt zu unterstützen und die Vorbereitung auf Prüfungen nachhaltig zu verbessern.

4. Projektumfeld

Das Projekt wird im Praktikumsbetrieb **PPI Media GmbH** in Hamburg durchgeführt. Das Unternehmen ist im Bereich Softwarelösungen und Dienstleistungen tätig und bietet eine moderne IT-Infrastruktur, die für die Umsetzung des Projekts genutzt werden kann.

Mir steht ein voll ausgestatteter Arbeitsplatz mit einem Entwicklungsrechner zur Verfügung. Auf diesem sind alle notwendigen Werkzeuge installiert, darunter:

- **IntelliJ IDEA** als Entwicklungsumgebung für das Backend,
- **Node.js** und eine IDE wie Visual Studio Code für das Frontend,
- **PostgreSQL** als relationale Datenbank sowie **pgAdmin** für die Administration,
- **Postman** für die Durchführung und Dokumentation von API-Tests,
- **draw.io** für die Erstellung von UML- und ER-Diagrammen,
- **Microsoft Word** für die Projektdokumentation.

Die Durchführung des Projekts erfolgt selbstständig, wobei mein Betreuer mir als Ansprechpartner bei fachlichen oder organisatorischen Fragen zur Verfügung steht. Für die Qualitätssicherung wird das Projekt nach Fertigstellung intern getestet.

Das Projekt wird vollständig im betrieblichen Umfeld realisiert und spiegelt praxisrelevante Anforderungen wider, die auch in zukünftigen Projekten und Kundenlösungen des Unternehmens Anwendung finden können.

5. Planung

5.1 Anforderungen

Funktionale Anforderungen:

- Registrierung und Login von Benutzern
- Verwaltung von Benutzerdaten (Passwortänderung, E-Mail)
- Bereitstellung eines zentralen Fragenpools, unterteilt in Themengebiete

- Abgabe von Antworten
- Automatische Auswertung der Antworten (richtig oder falsch)
- Anzeige einer Ergebnisübersicht
- Speicherung der Ergebnisse

Optionale Erweiterungen:

- Fortschrittsanzeige (z. B. Lernstatistik)
- Wiederholungslogik für falsch beantwortete Fragen
- Admin-Oberfläche zum Hinzufügen neuer Fragen

Nicht-funktionale Anforderungen:

- **Benutzerfreundlichkeit:** Die Anwendung soll ein klares, übersichtliches UI haben und responsiv auf Desktop, Tablet und Smartphone nutzbar sein.
- **Sicherheit:** Passwörter werden verschlüsselt gespeichert, Eingaben validiert.
- **Performance:** Die Beantwortung und Auswertung soll in < 500 ms erfolgen.
- **Zuverlässigkeit:** Anwendung läuft stabil und fehlerfrei, auch bei mehreren gleichzeitigen Nutzern.
- **Wartbarkeit:** Saubere Trennung von Frontend, Backend und Datenbank, gut dokumentierter Code.
- **Testbarkeit:** Unit-Tests im Backend, API-Tests mit Postman, Funktionstests im Frontend.

5.2 Use Cases

Use Case 1 – Benutzer registrieren / anmelden

- **Akteur:** Gast oder Benutzer
  Vorbedingung: Benutzer ist nicht angemeldet
  Ablauf:
- Benutzer gibt E-Mail und Passwort ein.
- System prüft Eingaben und erstellt ein Konto (Registrierung) oder meldet Benutzer an (Login).

- Benutzer gelangt zur Startseite.
  Nachbedingung: Benutzer ist eingeloggt und kann Fragen bearbeiten.

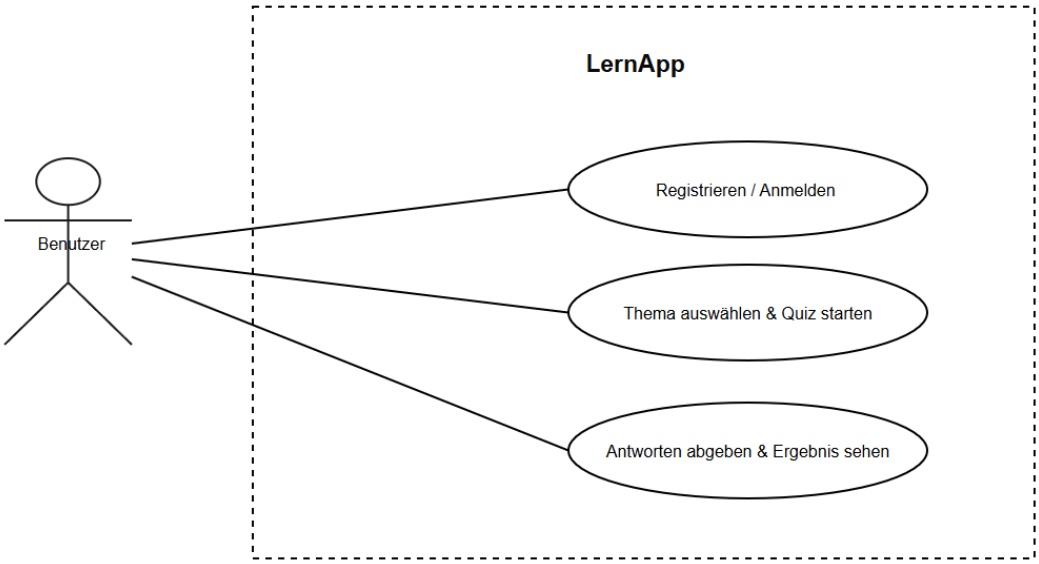
Use Case 2 – Quiz starten

- **Akteur:** Angemeldeter Benutzer
  Vorbedingung: Benutzer ist angemeldet
  Ablauf:
- Benutzer wählt auf der Startseite ein Themengebiet aus (z.B. Wirtschafts- und Sozialkunde, Programmierung).
- Das System lädt ausschließlich Fragen aus dem gewählten Themengebiet.
- Das Quiz wird gestartet.
  Nachbedingung: Eine themenspezifische Quiz-Session ist aktiv.

Use Case 3 – Antworten abgeben & Ergebnis sehen

- **Akteur:** Angemeldeter Benutzer
  Vorbedingung: Quiz läuft
  Ablauf:
- Benutzer beantwortet Fragen durch Auswahl von Antwortoptionen.
- System speichert die Antworten und wertet sie aus.
- Benutzer erhält Übersicht der richtigen und falschen Antworten.
  Nachbedingung: Ergebnisse sind gespeichert und für den Benutzer einsehbar.

Abbildung 1: Use-Case-Diagramm



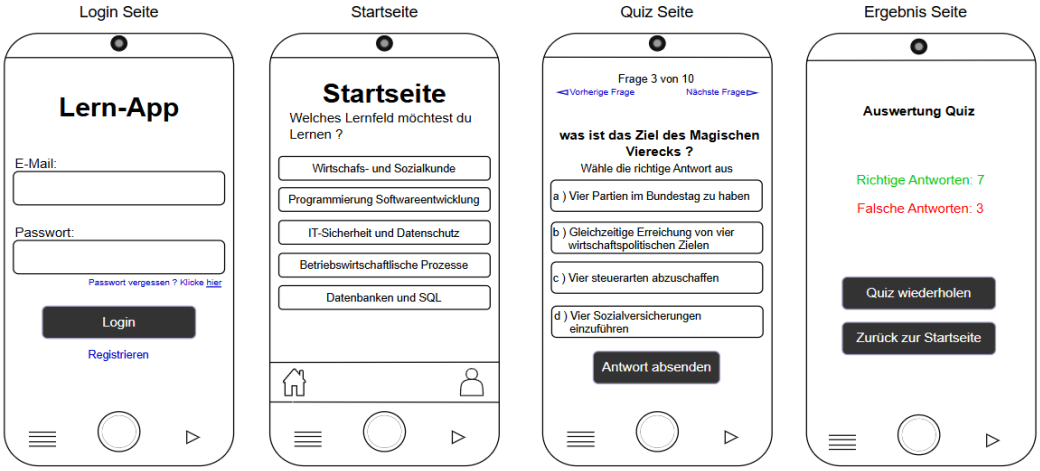
5.3 Mockups

Zur Veranschaulichung des geplanten Bedienkonzepts wurden einfache Mockups erstellt. Die Mockups zeigen die zentralen Ansichten der geplanten Anwendung:

- **Login-Seite**: Eingabe von E-Mail und Passwort sowie Möglichkeit zur Registrierung.
- **Startseite:** Nach erfolgreichem Login kann der Benutzer ein Themengebiet auswählen und das dazugehörige Quiz starten.
- **Quiz-Seite:** Anzeige der jeweiligen Frage mit mehreren Antwortmöglichkeiten, Auswahl einer Antwortoption und Absenden der Eingabe.
- **Ergebnis-Seite**: Übersicht über die Anzahl richtig und falsch beantworteter Fragen sowie Optionen zum Wiederholen oder zur Rückkehr zur Startseite.

Die Mockups dienen als Grundlage für die spätere Umsetzung im Frontend und stellen sicher, dass die Benutzeroberfläche einfach, übersichtlich und responsiv gestaltet wird.

Abbildung 2: Mockups



5.4 Zeitplanung

Die folgende Tabelle zeigt die geplante zeitliche Aufteilung des Projekts in einzelne Phasen. Die Gesamtprojektzeit beträgt 80 Stunden und entspricht damit dem im Projektantrag genehmigten Umfang. Die Verteilung berücksichtigt die Komplexität der Aufgaben und die technische Umsetzung in Frontende und Backend.

Projektschritt	Tätigkeiten	Zeit
Planung & Analyse	Definition der Projektziele, Analyse der Anforderungen, Erstellung von Use Cases und Mockups, Technologierecherche, Vorbereitung der Entwicklungsumgebung.	10 h
Architektur & Modellierung	Erstellung des ER- und Klassendiagramms, Definition der Softwarearchitektur und REST-API-Struktur, Aufbau der Projektstruktur (Frontend/Backend).	12 h
Backend-Implementierung	Entwicklung der REST-Schnittstellen (Spring Boot), Benutzerverwaltung, Datenbankanbindung (PostgreSQL), Implementierung der themenbasierten Fragenlogik.	20 h
Frontend-Implementierung	Umsetzung des Designs in React, Integration der API-Endpunkte, Themenauswahl, Quizlogik, Ergebnisanzeige.	20 h
Integration & Test	Verbindung von Frontend und Backend, Funktionstests, Fehlerbehebung, End-to-End-Überprüfung.	10 h
Projektdokumentation	Erstellung der Projektdokumentation, Integration von Diagrammen und Screenshots, Nachbearbeitung und Formatierung	8 h
Gesamt		**80 h**

6. Entwurf

6.1 Systemarchitektur

Die LernApp wird als dreischichtige Webanwendung umgesetzt, die aus Frontend, Backend und Datenbank besteht.
Diese Architektur trennt Darstellung, Logik und Datenhaltung klar voneinander und ermöglicht eine einfache Erweiterbarkeit sowie Wartbarkeit.

- **Frontend:** Das Frontend wurde mit React entwickelt und stellt die Benutzeroberfläche der Anwendung dar.
  Hier erfolgen die Interaktionen des Benutzers, wie Anmeldung, Themenauswahl und das Beantworten der Quizfragen.
  Das Frontend kommuniziert über HTTP(S)-Anfragen mit dem Backend.

- **Backend:** Das Backend wurde mit Spring Boot (Java) umgesetzt und bildet die Geschäftslogik der Anwendung ab.
  Es verarbeitet Anfragen des Frontends, validiert Eingaben, ruft Daten aus der Datenbank ab und bereitet die Ergebnisse für die Darstellung auf.
  Über eine REST-API werden JSON-basierte Daten zwischen Frontend und Backend ausgetauscht.

- **Datenbank:** Die Datenhaltung erfolgt in einer PostgreSQL-Datenbank.
  Dort werden alle relevanten Informationen wie Benutzer, Fragen, Themen und Ergebnisse gespeichert.
  Durch die klare Trennung von Daten- und Anwendungsschicht wird eine hohe Datenkonsistenz und -sicherheit gewährleistet.

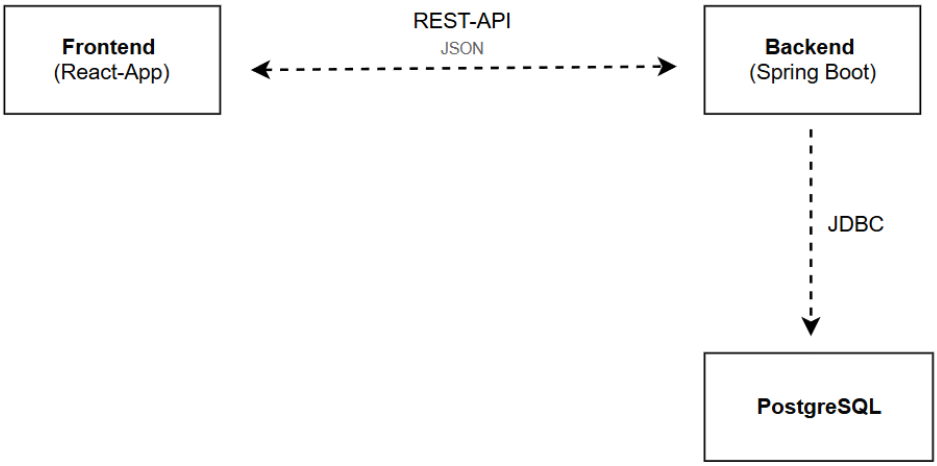
Kommunikation:
Das Frontend sendet Anfragen an das Backend über definierte REST-Endpunkte

(z. B. /api/login, /api/questions/topic}

Das Backend verarbeitet diese Anfragen und antwortet mit strukturierten JSON-Daten, die das Frontend anschließend rendert.

Diese Architektur ermöglicht eine flexible Weiterentwicklung, z. B. durch spätere Integration neuer Themenbereiche oder die Anbindung zusätzlichen Frontens (z. B. mobile Apps).

Abbildung 3: Systemarchitektur



6.2 Datenmodell

Das Datenmodell der LernApp wurde auf Basis der funktionalen Anforderungen erstellt.
Es beschreibt die Struktur der in der PostgreSQL-Datenbank gespeicherten Daten sowie die Beziehungen zwischen den einzelnen Entitäten.

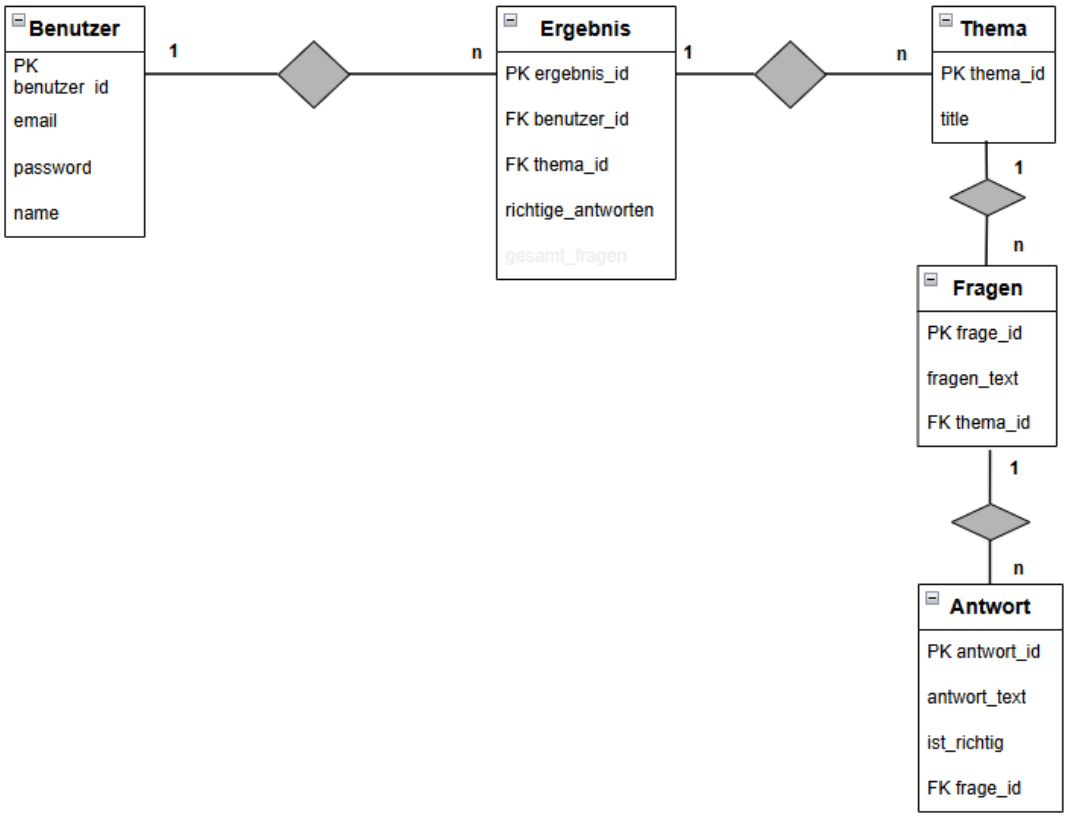
Ziel des Datenmodells ist es, eine konsistente und nachvollziehbare Speicherung von Benutzerdaten, Fragen, Themen und Ergebnissen sicherzustellen.
Dabei wurden Redundanzen vermieden und logische Abhängigkeiten über Primär- und Fremdschlüssel definiert.

Das nachfolgende **ER-Diagramm (Entity-Relationship-Diagramm)** zeigt die zentralen Entitäten der Anwendung sowie ihre Beziehungen untereinander:

- **Benutzer :** Speichert die Zugangsdaten und den Namen des Lernenden.
- **Thema :** Enthält die verschiedenen Themengebiete der Prüfung.
- **Fragen :** Beinhaltet die einzelnen Quizfragen, die einem bestimmten Thema zugeordnet sind.
- **Antwort :** Repräsentiert mögliche Antworten zu einer Frage.
- **Ergebnis :** Dokumentiert die vom Benutzer erzielten Punkte und speichert die Zuordnung von Benutzer und Thema.

Jede Frage ist genau einem Thema zugeordnet, und jede Antwort gehört zu genau einer Frage.
Benutzer können mehrere Ergebnisse besitzen, die jeweils einem Thema zugeordnet sind.

Abbildung 4: ER-Diagramm



6.3 Klassendiagramm

Das Klassendiagramm stellt den objektorientierten Aufbau der Anwendung im Backend dar.
Es zeigt die wichtigsten Klassen, deren Attribute und Beziehungen untereinander.
Die Umsetzung basiert auf dem Spring-Boot-Framework und folgt dem Prinzip der Model-View-Controller-Architektur (MVC).

Im Fokus stehen hier die **Model-Klassen**, welche die Entitäten der Anwendung repräsentieren und direkt mit der Datenbank (PostgreSQL) verknüpft sind.

Die Klassen wurden so gestaltet, dass sie den im ER-Diagramm dargestellten Entitäten entsprechen und eine konsistente Abbildung der Datenstruktur gewährleisten.

Die wichtigsten Klassen und ihre Aufgaben:

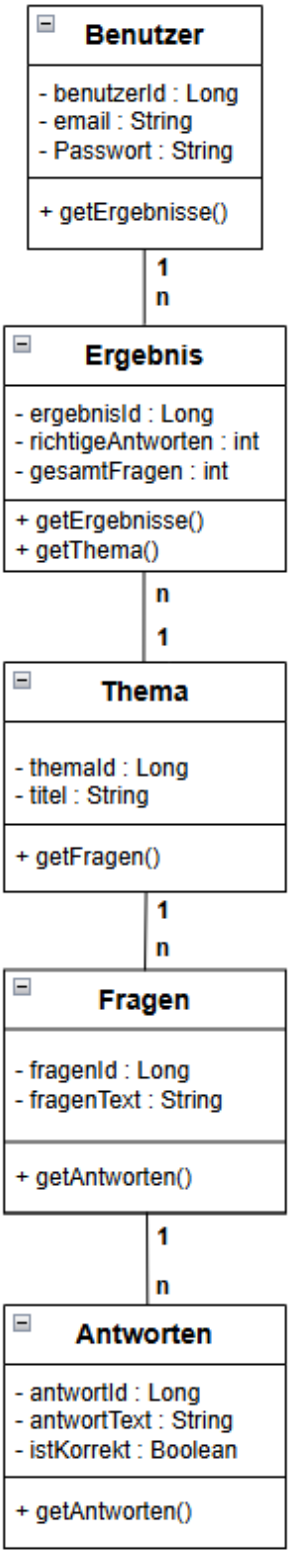
- **Benutzer:** Repräsentiert den Benutzer der Anwendung. Speichert Login-Daten (E-Mail, Passwort) und dient als Basis für die Zuordnung der erzielten Ergebnisse.
- **Thema:** Stellt ein Themengebiet dar, zu dem Fragen hinterlegt werden.
- **Frage:** Enthält den Fragetext, eine Referenz zum zugehörigen Thema sowie die zugehörigen Antwortmöglichkeiten.
- **Antwort:** Repräsentiert eine mögliche Antwort auf eine Frage und kennzeichnet, ob sie korrekt ist.
- **Ergebnis:** Speichert die Anzahl richtig beantworteter Fragen eines Benutzers in einem bestimmten Thema.

Die Beziehungen zwischen den Klassen spiegeln die Verknüpfungen aus dem Datenmodell wider:

- Ein **Benutzer** kann mehrere **Ergebnise** besitzen (1: n).
- Ein **Thema** kann mehrere **Fragen** und mehrere **Ergebnisse** enthalten (1: n).
- Eine **Frage** besitzt mehrere **Antworten** (1: n).

  Das folgende Klassendiagramm zeigt die Struktur der wichtigsten Modellklassen im backend:

Abbildung 5: Klassendiagramm



7. Implementierung

In diesem Kapitel wird die technische Umsetzung der LernApp beschrieben. Die Implementierung gliedert sich in drei Bereiche: Backend (Spring Boot), Datenbank (PostgreSQL) und Frontend (React). Detailansichten von Klassen, REST-Endpunkten, Konfigurationen und Screenshots befinden sich im Anhang.

7.1 Backend-Implementierung

Das Backend wurde mit **Java und Spring Boot** umgesetzt und stellt die REST-Schnittstelle für das Frontend bereit. Die Anwendung ist so strukturiert, dass fachliche Verantwortlichkeiten klar getrennt sind.

7.1.1 Architektur und Projektstruktur

Die Backend-Anwendung ist in mehrere Pakete gegliedert, zum Beispiel für Domänenklassen (Entitäten), Datenzugriffe (Repositories), Webschicht (Controller, DTOs) und Konfigurationen.
Dadurch wird eine saubere Trennung zwischen Geschäftslogik, Persistenz und Präsentationslogik erreicht.

Verweis: Projektstruktur und Paketübersicht siehe AnhangA 2.

7.1.2 CORS-Konfiguration und REST-Schnittstelle

Da das Frontend während der Entwicklung unter http://localhost:3000 und das Backend unter http://localhost:8086 läuft, musste eine CORS-Konfiguration eingerichtet werden.
Diese Konfiguration erlaubt dem React-Frontend, die benötigten HTTP-Methoden (z. B. GET und POST) gegen die REST-API des Backends aufzurufen.

Die REST-Schnittstelle stellt unter anderem folgende Funktionen bereit:

- Abruf aller verfügbaren Themen
- Abruf der Fragen inklusive Antwortmöglichkeiten zu einem gewählten Thema
- Entgegennahme und Speicherung von Quiz-Ergebnissen

Verweis: CORS-Konfiguration und Übersicht der REST-Endpunkte siehe Anhang A3 und A3.

7.1.3 Verarbeitung und Speicherung der Ergebnisse

Nach Abschluss eines Quiz sendet das Frontend das Ergebnis (z. B. Anzahl richtiger Antworten, Gesamtscore und gewähltes Thema) an das Backend.
Dort wird das Ergebnis über einen REST-Endpunkt entgegengenommen, in eine Ergebnis-Entität überführt und in der Datenbank gespeichert.

Verweis: Ablauf der Ergebnisspeicherung und Beispiel-Request siehe Anhang A4.

7.2 Datenbank-Implementierung

Für die Persistenz der Daten kommt eine **PostgreSQL-Datenbank** zum Einsatz. Die Datenbank bildet die in der Analyse und im Entwurf definierten Entitäten ab.

7.2.1 Tabellen und Beziehungen

Die zentrale Tabellenstruktur besteht aus folgenden Tabellen:

- **thema** – verwaltet die verschiedenen Themengebiete (z. B. Programmierung, IT-Sicherheit, WISO)
- **frage** – speichert die einzelnen Quizfragen, jeweils einem Thema zugeordnet
- **antwort** – enthält die Antwortmöglichkeiten zu den Fragen und kennzeichnet, welche Antwort korrekt ist
- **benutzer** – (optional) zur Speicherung von Benutzerdaten
- **ergebnis** – dokumentiert abgeschlossene Quizdurchläufe (z. B. Thema, Anzahl richtiger Antworten)

Zwischen den Tabellen bestehen 1:n-Beziehungen, zum Beispiel:

- Ein Thema kann mehrere Fragen besitzen
- Eine Frage besitzt mehrere Antworten
- Ein Benutzer kann mehrere Ergebnisse haben

Verweis: ER-Diagramm und Tabellenübersicht in PostgreSQL siehe Anhang B1.

7.2.2 Initial Befüllung mit Fragen und Antworten

Zur Vorbereitung der Anwendung wurde die Datenbank mit einem umfangreichen Fragen- und Antwortkatalog befüllt. Pro Thema wurden mehrere Fragen angelegt, wobei jede Frage genau vier Antwortmöglichkeiten besitzt (eine richtige, drei falsche).

Die Einträge wurden über SQL-Skripte in pgAdmin eingefügt. Dies erleichtert die Reproduzierbarkeit und eine mögliche Erweiterung des Fragenpools.

Verweis: Auszüge der verwendeten SQL-Insert-Skripte und Screenshots aus pgAdmin siehe Anhang B2.

7.3 Frontend-Implementierung

Das Frontend wurde mit **React** umgesetzt und bietet dem Benutzer eine einfache, auf das Wesentliche reduzierte Oberfläche zur Durchführung von themenbasierten Quizrunden.

7.3.1 Aufbau und Komponenten

Die Frontend-Anwendung ist in einzelne Komponenten aufgeteilt, zum Beispiel:

- **Themenauswahl** – zeigt alle verfügbaren Themen an und ermöglicht dem Benutzer die Auswahl eines Themengebiets
- **Quizansicht** – zeigt nacheinander die Fragen und die dazugehörigen Antwortmöglichkeiten
- **Ergebnisanzeige** – fasst am Ende die Anzahl der richtigen Antworten zusammen

Die Kommunikation mit dem Backend erfolgt über eine zentrale API-Datei, die die REST-Aufrufe kapselt. Dadurch bleibt die UI-Logik von den technischen Details der HTTP-Kommunikation getrennt.

Verweis: Projektstruktur und API-Anbindung im Frontend siehe Anhang C1 und C2.

7.3.2 Ablauf der Quiz-Logik

Nach dem Start der Anwendung lädt das Frontend zunächst die Liste der verfügbaren Themen über die REST-Schnittstelle. Der Benutzer wählt ein Thema, woraufhin die zugehörigen Fragen aus dem Backend geladen werden.

Die Quizansicht zeigt jeweils eine Frage mit ihren vier möglichen Antworten an. Nach der Auswahl einer Antwort wird intern gespeichert, ob diese korrekt war. Am Ende des Fragekatalogs wird die Gesamtanzahl der richtigen Antworten berechnet und dem Benutzer angezeigt.

Zusätzlich wird das Ergebnis an das Backend übermittelt, sodass die Quizdurchläufe in der Datenbank dokumentiert sind.

Verweis: Screenshots der Themenauswahl, Quizansicht und Ergebnisanzeige siehe Anhang D1.

8 Testphase

Die Testphase dient der Sicherstellung der korrekten Funktionalität der LernApp. Es wurden verschiedene Testmethoden angewendet, um sowohl die Backend-API als auch die Frontend-Funktionen realistisch zu prüfen. Ziel war es, Fehler frühzeitig zu erkennen und die Anwendung stabil für die Nutzung vorzubereiten.

Alle Screenshots und Testergebnisse befinden sich im Anhang.

8.1 Vorgehensweise bei den Tests

Die Tests wurden in drei Kategorien durchgeführt:

1. **Modultests (Backend)**
   - Überprüfung einzelner REST-Endpunkte
   - Testen der Datenbankabfragen
   - Kontrolle der JSON-Ausgaben
     (siehe Anhang D1–D3)
2. **Integrationstests (Backend ↔ Frontend)**
   - Kommunikation zwischen React und Spring Boot
   - Prüfung der CORS-Freigabe
   - Abruf von Themen und Fragen über das Frontend
     (siehe Anhang D4)
3. **Funktionstests (UI-Tests im Browser)**

- Manuelles Durchklicken der vollständigen Quizstrecke
- Testen der Benutzerführung
- Prüfung der Ergebnisanzeige
  (siehe Anhang)

8.2 Testfälle

Im Rahmen des Projekts wurden folgende wesentliche Testfälle definiert und erfolgreich ausgeführt:

Testfall 1: Backend erreichbar (Ping-Test)

Ziel: Sicherstellen, dass das Backend lauffähig ist und Anfragen verarbeitet.
Durchführung: Aufruf von /api/ping im Browser.
Erwartetes Ergebnis: Antwort „ping“.
Status: Erfolgreich.
(siehe Anhang)

Testfall 2: Themenliste wird korrekt geladen

Ziel: Prüfen, ob die Themen aus der Datenbank abgerufen werden können.
Durchführung: Starten des Frontends → Themen werden automatisch geladen.
Erwartetes Ergebnis: Themen „Programmierung“, „IT-Sicherheit“ und „Wirtschaft & Soziales“ werden angezeigt.
Status: Erfolgreich.
(siehe Anhang)

Testfall 3: Fragen eines Themas abrufen

Ziel: Sicherstellen, dass zu einem Thema die korrekten Fragen und Antworten geliefert werden.
Durchführung: Auswahl eines Themas im Frontend → REST-Aufruf /api/fragen?themaId=….
Erwartetes Ergebnis: JSON-Liste mit vollständigen Frage- und Antwortdaten.
Status: Erfolgreich.
(siehe Anhang)

Testfall 4: Quizdurchführung (Frontend)

Ziel: Überprüfung der Quiz-Logik.
Durchführung:

- Thema auswählen
- „Quiz starten“
- Alle Fragen nacheinander beantworten

Erwartetes Ergebnis:

- Jede Frage wird korrekt angezeigt
- Antwortauswahl funktioniert
- Fortschritt (z. B. „Frage 1 von 10“) wird korrekt dargestellt
- Am Ende wird das Ergebnis korrekt berechnet

Status: Erfolgreich.
(siehe Anhang)

Testfall 5: Ergebnis speichern im Backend

Ziel: Überprüfung der Speicherung eines Quizdurchgangs.
Durchführung: Quiz vollständig durchspielen → Ergebnis wird automatisch gesendet.
Erwartetes Ergebnis: Ein neuer Datensatz erscheint in der Tabelle ergebnis.
Status: Erfolgreich.
(siehe Anhang)

Testfall 6: Fehlerbehandlung (z. B. Backend offline)

Ziel: Sicherstellen, dass Fehler sinnvoll an den Benutzer kommuniziert werden.
Durchführung: Backend deaktiviert → Frontend starten.
Erwartetes Ergebnis: Benutzer erhält eine Fehlermeldung („Themen konnten nicht geladen werden.“).
Status: Erfolgreich.
(siehe Anhang)

8.3 Testauswertung und Qualitätssicherung

Alle durchgeführten Testfälle wurden erfolgreich abgeschlossen.
Die Anwendung zeigte im Zusammenspiel zwischen Backend, Datenbank und Frontend ein konsistentes Verhalten. Die wichtigsten Erkenntnisse:

- Die REST-Schnittstelle liefert stabile Datenstrukturen
- Alle Quizfragen werden korrekt angezeigt
- Die Berechnung der Punkte erfolgt fehlerfrei
- Fehler (z. B. fehlende Verbindung) werden verständlich abgefangen
- Die Integration zwischen React und Spring Boot funktioniert zuverlässig
- Die Datenbank speichert neue Ergebnisse ohne Probleme

Damit erfüllt das System die definierten funktionalen Anforderungen vollständig.

8.4 Zusammenfassung

Die Testphase bestätigt, dass die LernApp technisch stabil und funktional einsatzbereit ist.
Die wichtigsten Funktionen wurden erfolgreich verifiziert – vom Abrufen der Themen und Fragen über die Durchführung des Quiz bis hin zur Speicherung der Ergebnisse.

9 Wirtschaftlichkeitsbetrachtung

Die Entwicklung der webbasierten LernApp wurde im Rahmen des IHK-Abschlussprojekts durchgeführt. Obwohl das Projekt keinen direkten finanziellen Auftrag hatte, lässt sich dennoch eine wirtschaftliche Betrachtung durchführen. Ziel ist es, den Nutzen der Anwendung im Verhältnis zum Aufwand zu bewerten und deren Mehrwert für den Ausbildungsbetrieb darzustellen.

9.1 Ausgangslage

Im Unternehmen existierte keine digitale Lösung zur strukturierten Prüfungsvorbereitung für Auszubildende im Bereich Fachinformatik. Externe Plattformen sind oft kostenpflichtig, unstrukturiert oder nicht auf die betrieblichen Anforderungen zugeschnitten.

Durch die Eigenentwicklung können Inhalte gezielt auf die relevanten Prüfungsgebiete angepasst und jederzeit erweitert werden.

9.2 Aufwand

Der Aufwand setzt sich aus den folgenden Bereichen zusammen:

- **Analyse und Planung**
  Strukturierung der Anforderungen, Erstellen von Diagrammen und Mockups
- **Implementierung Backend**
  Entwicklung der REST-API, Datenbankmodellierung und Persistenzlogik
- **Implementierung Frontend**
  Aufbau der Benutzeroberfläche, API-Anbindung, Darstellung der Quizlogik
- **Testen und Fehlerbehebung**
  Funktions-, Integrations- und Oberflächentests
- **Dokumentation**
  Erstellen der Projektdokumentation gemäß IHK-Vorgaben

Da das Projekt im Rahmen der Ausbildung durchgeführt wurde, entstanden **keine direkten finanziellen Kosten**. Die benötigte Hard- und Software war bereits vorhanden.

Die gesamte Projektlaufzeit beträgt gemäß Projektantrag **80 Stunden**, verteilt auf die genannten Tätigkeiten.

9.3 Nutzen für den Ausbildungsbetrieb

Die LernApp bietet dem Betrieb mehrere Vorteile:

● 1. Verbesserung der Prüfungsvorbereitung

Auszubildende können selbstständig und strukturiert Lerninhalte wiederholen, insbesondere themenbasiert (Programmierung, IT-Sicherheit, WISO).

● 2. Wiederverwendbarkeit und Erweiterbarkeit

Neue Fragen oder Themen lassen sich einfach über SQL oder Erweiterungen im Backend ergänzen.
Damit kann die App langfristig genutzt und verbessert werden.

● **3. Senkung externer Kosten**

Externe Lernplattformen oder zusätzliche Lernmaterialien können entfallen.
Die App ist vollständig betriebseigen und verursacht keine Lizenzkosten.

● **4. Einheitliche Lernstruktur**

Alle Auszubildenden lernen auf der gleichen Grundlage.
Dies erleichtert interne Schulungen und Wissenstransfer.

● **5. Förderung der internen Digitalisierung**

Das Projekt stärkt die interne IT-Kompetenz und zeigt, wie moderne Web-Anwendungen im Betrieb umgesetzt werden können.

9.4 Wirtschaftliches Fazit

Der betriebliche Nutzen der Anwendung übersteigt den Entwicklungsaufwand deutlich.
Die LernApp ist kostenfrei nutzbar, flexibel erweiterbar und unterstützt Auszubildende direkt bei der Vorbereitung auf ihre Abschlussprüfung.

Da keine externen Dienste erforderlich sind und die App vollständig intern betrieben wird, entstehen auch zukünftig keine laufenden Kosten.

Insgesamt ist die LernApp somit wirtschaftlich sinnvoll und nachhaltig einsetzbar.

10 Fazit und Reflexion

Im Rahmen dieses Projekts wurde eine vollständige, themenbasierte LernApp entwickelt, die Auszubildenden eine strukturierte und digitale Möglichkeit zur Prüfungsvorbereitung bietet. Durch die Kombination aus einem Spring-Boot-Backend, einer PostgreSQL-Datenbank und einem React-Frontend konnte eine moderne Webanwendung umgesetzt werden, die flexibel erweiterbar und intuitiv bedienbar ist.

Während der Umsetzung konnten alle im Projektantrag definierten Ziele vollständig erreicht werden. Die Anwendung ermöglicht:

- das Abrufen von Themen,

- das Durchführen von Quizfragen,
- die Auswertung der Antworten
- sowie die Speicherung der Ergebnisse.

Die technische Umsetzung verlief größtenteils planmäßig. Herausforderungen ergaben sich vor allem in der Abstimmung zwischen Frontend und Backend, insbesondere bei der Datenübertragung (CORS, API-Struktur) und beim Aufbau der Datenbank. Diese konnten jedoch durch systematisches Testen, gezielte Fehlersuche und Anpassungen im Backend erfolgreich gelöst werden.

Durch das Projekt konnte ich meine Kenntnisse in Java, Spring Boot, React und SQL vertiefen und die vollständige Umsetzung einer Webanwendung von der Planung bis zur Dokumentation praktisch erleben. Besonders hilfreich war die iterative Vorgehensweise, bei der einzelne Komponenten nacheinander entwickelt und getestet wurden.

Die LernApp bietet dem Ausbildungsbetrieb einen klaren Mehrwert, da sie langfristig genutzt, angepasst und erweitert werden kann. Neue Themen oder Fragen lassen sich jederzeit hinzufügen, ohne den grundlegenden Aufbau verändern zu müssen.

Insgesamt betrachte ich das Projekt als erfolgreich. Ich konnte nicht nur meine fachlichen Fähigkeiten stärken, sondern auch wichtige Erfahrungen in der Softwareentwicklung sammeln – insbesondere im Bereich der Schnittstellenentwicklung, der Fehleranalyse und der strukturierten Projektplanung.

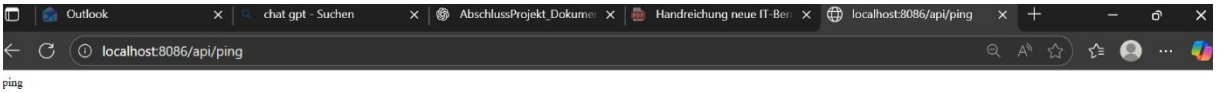
Die Umsetzung hat gezeigt, wie wichtig klare Anforderungen, kontinuierliches Testen und eine saubere Dokumentation sind. Das Projekt bildet für mich einen wertvollen Abschluss meiner praktischen Ausbildung und eine gute Grundlage für zukünftige berufliche Projekte.

Anhang

Anhang A – Backend

A1 Backend-Start & Ping-Test

```
01:24:33: Executing ':de.abdul.ihk.lernapp.LernappApplication.main()'…

> Task :compileJava UP-TO-DATE
> Task :processResources UP-TO-DATE
> Task :classes UP-TO-DATE

> Task :de.abdul.ihk.lernapp.LernappApplication.main()

  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/

 :: Spring Boot ::                (v3.5.8)
```

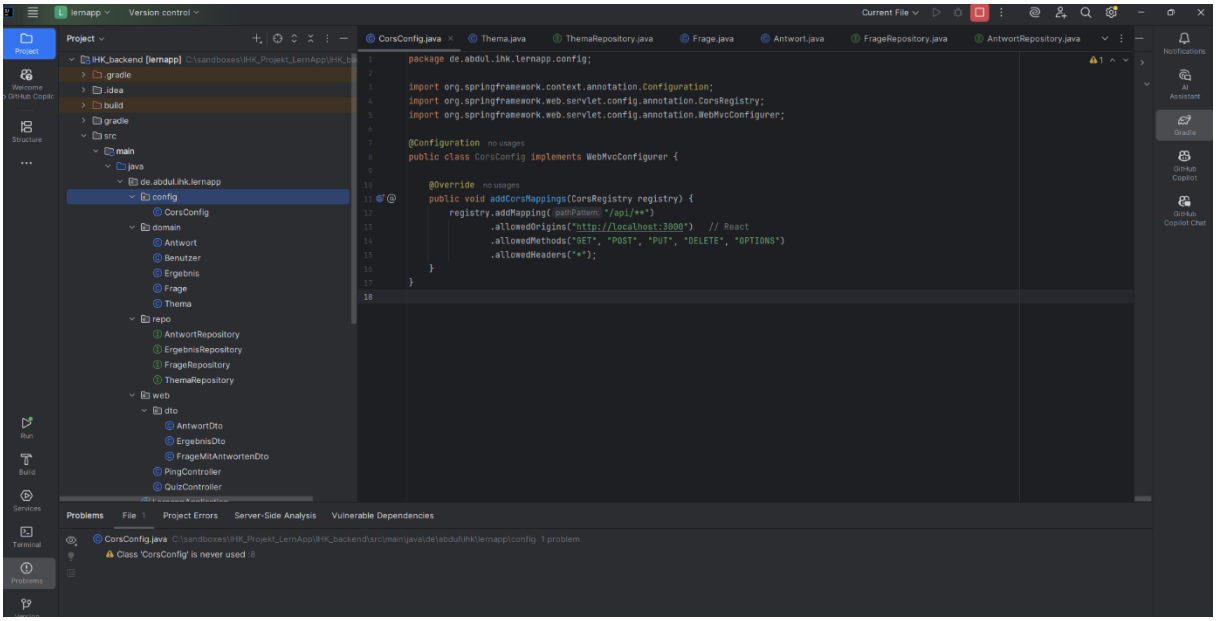


ping

A2 Start und Ausführung des Backends

```
2025-11-28T10:42:17.078+01:00  INFO 28944 --- [           main] o.s.c.c.c.[Tomcat].[localhost].[/]       : Initializing Spring embedded WebApplicationContext
2025-11-28T10:42:17.080+01:00  INFO 28944 --- [           main] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in 2803 ms
2025-11-28T10:42:17.352+01:00  INFO 28944 --- [           main] o.hibernate.jpa.internal.util.LogHelper  : HHH000204: Processing PersistenceUnitInfo [name: default]
2025-11-28T10:42:17.466+01:00  INFO 28944 --- [           main] org.hibernate.Version                    : HHH000412: Hibernate ORM core version 6.6.36.Final
2025-11-28T10:42:17.532+01:00  INFO 28944 --- [           main] o.h.c.internal.RegionFactoryInitiator    : HHH000026: Second-level cache disabled
2025-11-28T10:42:18.090+01:00  INFO 28944 --- [           main] o.s.o.j.p.SpringPersistenceUnitInfo      : No LoadTimeWeaver setup: ignoring JPA class transformer
2025-11-28T10:42:18.155+01:00  INFO 28944 --- [           main] com.zaxxer.hikari.HikariDataSource       : HikariPool-1 - Starting...
2025-11-28T10:42:18.763+01:00  INFO 28944 --- [           main] com.zaxxer.hikari.pool.HikariPool        : HikariPool-1 - Added connection org.postgresql.jdbc.PgConnection@12cd129c
2025-11-28T10:42:18.769+01:00  INFO 28944 --- [           main] com.zaxxer.hikari.HikariDataSource       : HikariPool-1 - Start completed.
2025-11-28T10:42:18.911+01:00  INFO 28944 --- [           main] org.hibernate.orm.connections.pooling    : HHH10001005: Database info:
        Database JDBC URL [Connecting through datasource 'HikariDataSource (HikariPool-1)']
        Database driver: undefined/unknown
        Database version: 16.8
        Autocommit mode: undefined/unknown
        Isolation level: undefined/unknown
        Minimum pool size: undefined/unknown
        Maximum pool size: undefined/unknown
2025-11-28T10:42:20.860+01:00  INFO 28944 --- [           main] o.h.e.t.j.p.i.JtaPlatformInitiator       : HHH000489: No JTA platform available (set 'hibernate.transaction.jta.platform' to enable JTA platform inte
gration)
2025-11-28T10:42:21.073+01:00  INFO 28944 --- [           main] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit 'default'
2025-11-28T10:42:21.423+01:00  WARN 28944 --- [           main] JpaBaseConfiguration$JpaWebConfiguration : spring.jpa.open-in-view is enabled by default. Therefore, database queries may be performed during view re
ndering. Explicitly configure spring.jpa.open-in-view to disable this warning
2025-11-28T10:42:22.429+01:00  INFO 28944 --- [           main] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat started on port 8086 (http) with context path '/'
2025-11-28T10:42:22.444+01:00  INFO 28944 --- [           main] d.abdul.ihk.lernapp.LernappApplication   : Started LernappApplication in 9.331 seconds (process running for 10.424)
<==========--> 80% EXECUTING [45s]
> :bootRun
```

A3 CORS-Konfiguration



A4 Test der REST-Schnittstellen (Postman)

pgAdmin 4

File Object Tools Edit View Window Help

Welcome | lernapp/postgres... | lernapp/postgres@PostgreSQL 16*

lernapp/postgres@PostgreSQL 16

No limit

Query | Query History

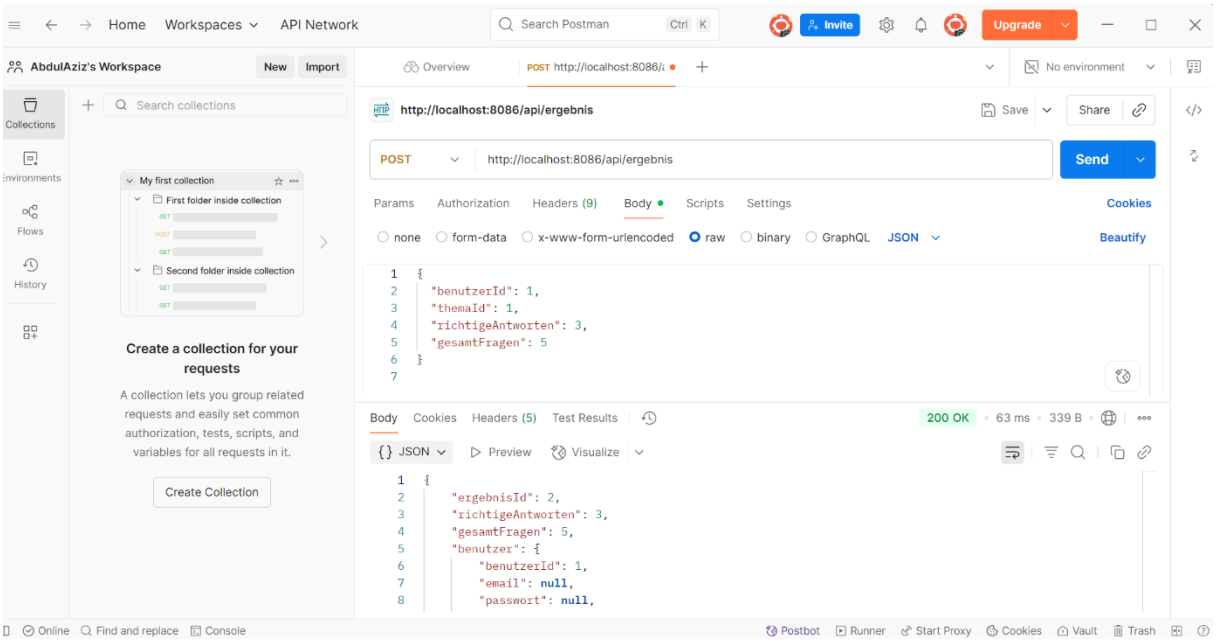
Execute script F5

Scratch Pad

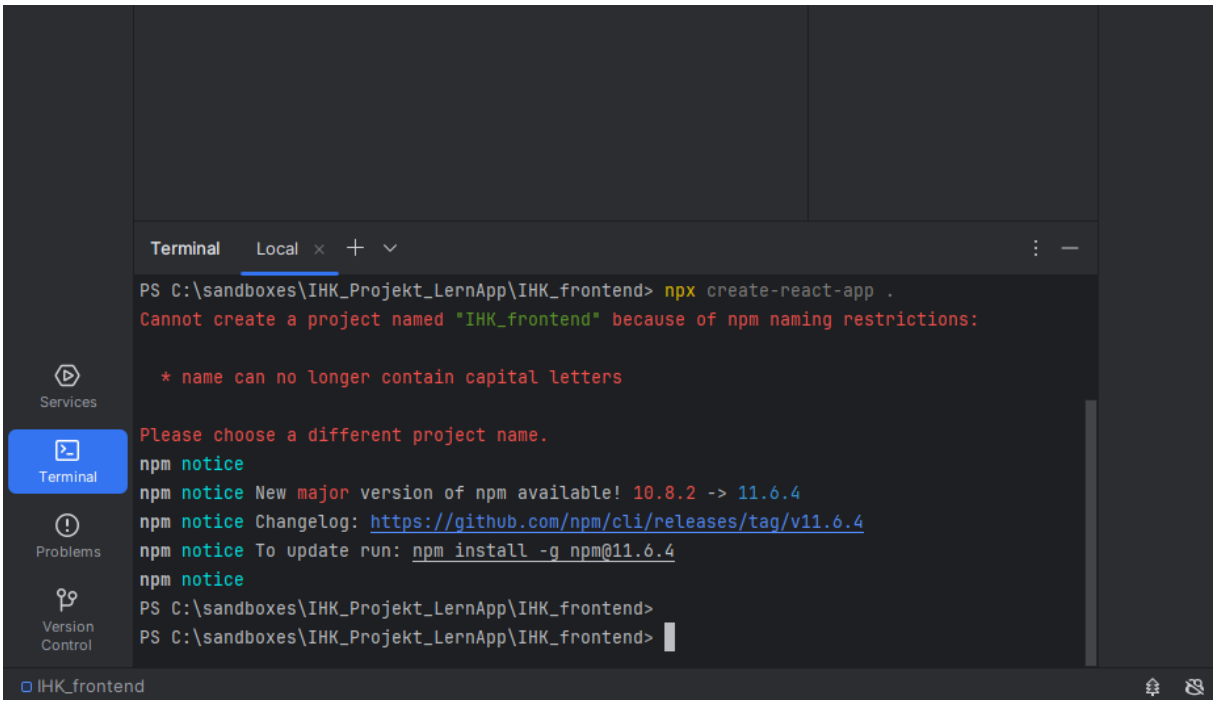
```
INSERT INTO thema(titel) VALUES
('Wirtschaft & Soziales'),
('Programmierung'),
('IT-Sicherheit');

INSERT INTO frage(frage_text, thema_id) VALUES
('Was ist ein Interface in Java?', 2),
('Welche Aussage zur DSGVO ist korrekt?', 1);

INSERT INTO antwort(antwort_text, ist_korrekt, frage_id) VALUES
('Vertrag zwischen Modulen', false, 1),
('Typdefinition von Methoden ohne Implementierung', true, 1),
('DSGVO gilt nur für Behörden', false, 2),
('DSGVO regelt Verarbeitung personenbezogener Daten in der EU', true, 2)
```

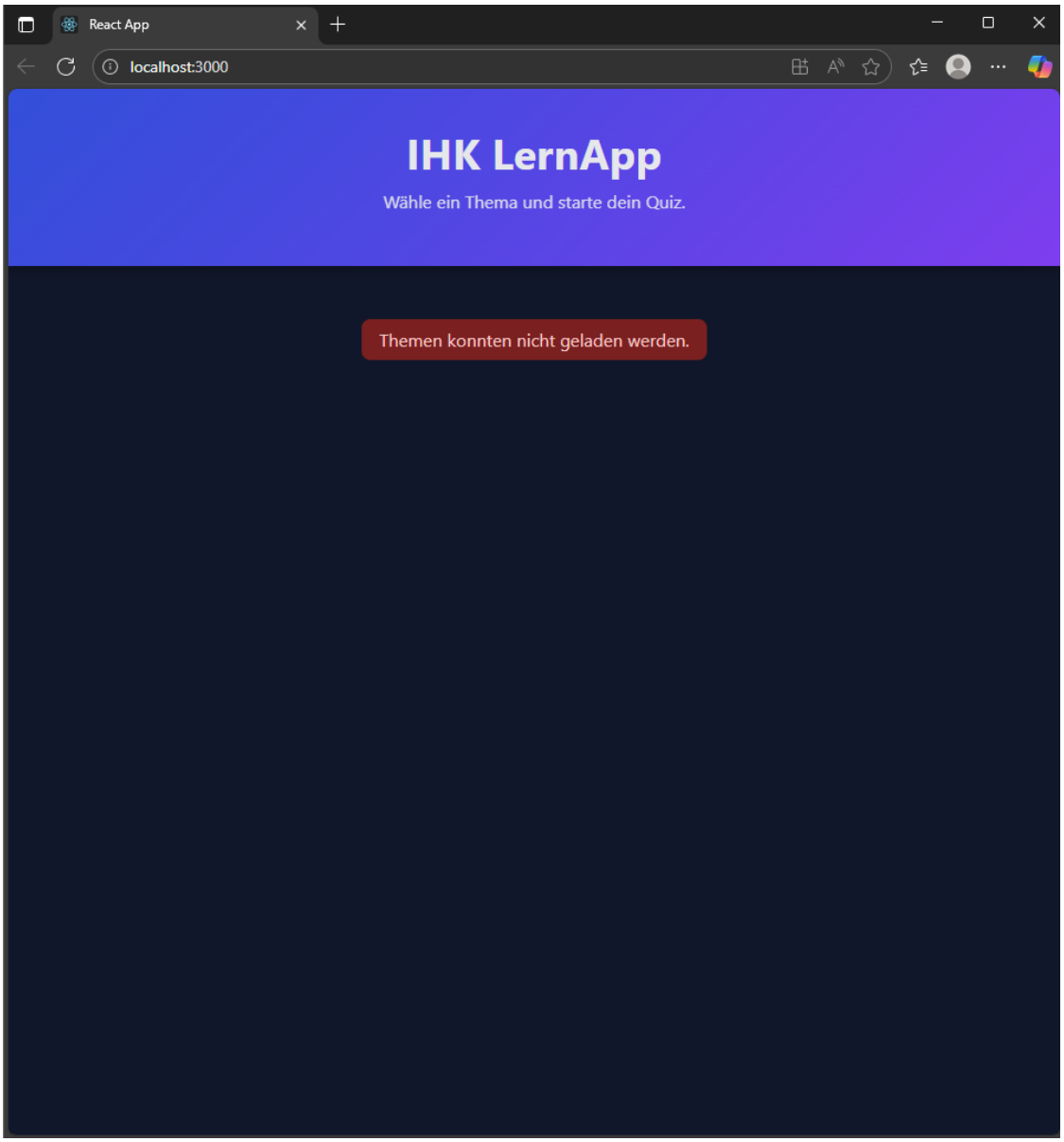
Data Output | Messages | Notifications

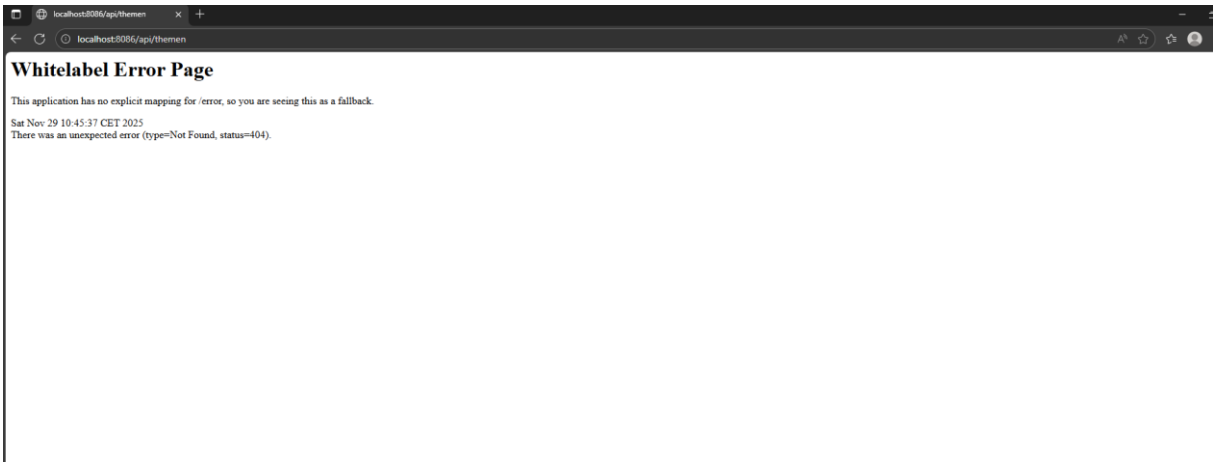
```
INSERT 0 4

Query returned successfully in 93 msec.
```


Home
Workspaces
API Network
Search Postman
Invite
Upgrade
AbdulAziz's Workspace
New
Import
Overview
POST http://localhost:8086/
No environment
Collections
Environments
Flows
History
Search collections
My first collection
First folder inside collection
Second folder inside collection
Create a collection for your requests
A collection lets you group related requests and easily set common authorization, tests, scripts, and variables for all requests in it.
Create Collection
http://localhost:8086/api/ergebnis
Save
Share
POST
http://localhost:8086/api/ergebnis
Send
Params
Authorization
Headers (9)
Body
Scripts
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Beautify
{
"benutzerId": 1,
"themaId": 1,
"richtigeAntworten": 3,
"gesamtFragen": 5
}
Body
Cookies
Headers (5)
Test Results
200 OK
63 ms
339 B
JSON
Preview
Visualize
{
"ergebnisId": 2,
"richtigeAntworten": 3,
"gesamtFragen": 5,
"benutzer": {
"benutzerId": 1,
"email": null,
"passwort": null,
Online
Find and replace
Console
Postbot
Runner
Start Proxy
Cookies
Vault
Trash

A5 Fehleranalyse & Debugging



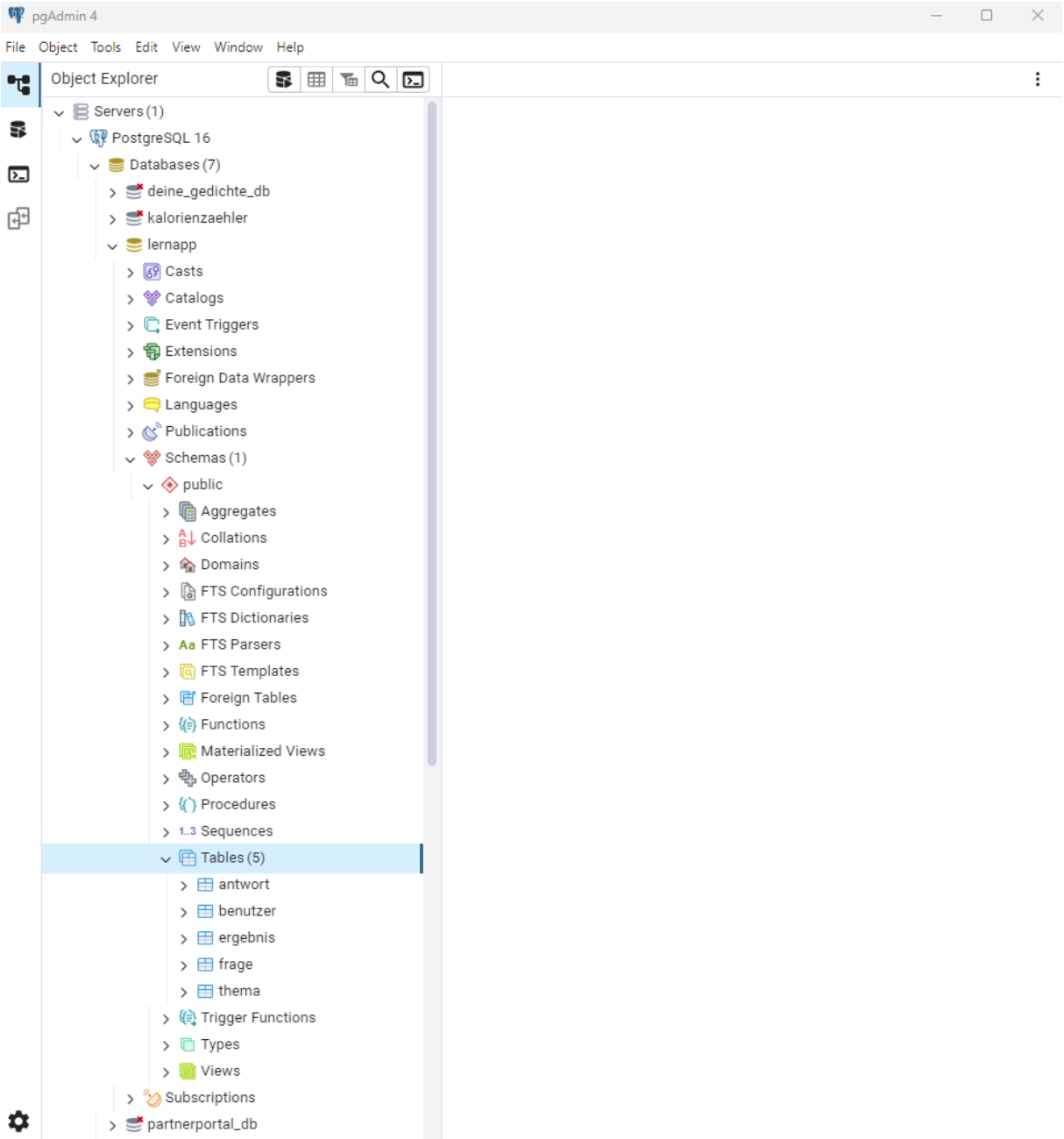

React App
localhost:3000
IHK LernApp
Wähle ein Thema und starte dein Quiz.
Themen konnten nicht geladen werden.


localhost:8086/api/themen
localhost:8086/api/themen
Whitelabel Error Page
This application has no explicit mapping for /error, so you are seeing this as a fallback.
Sat Nov 29 10:45:37 CET 2025
There was an unexpected error (type=Not Found, status=404).

```
313

Data Output   Messages   Notifications

ERROR:  Einfügen oder Aktualisieren in Tabelle »antwort« verletzt Fremdschlüssel-Constraint »antwort_frage_id_fkey«
Schlüssel (frage_id)=(30) ist nicht in Tabelle »frage« vorhanden.

FEHLER:  Einfügen oder Aktualisieren in Tabelle »antwort« verletzt Fremdschlüssel-Constraint »antwort_frage_id_fkey«
SQL state: 23503
Detail: Schlüssel (frage_id)=(30) ist nicht in Tabelle »frage« vorhanden.
```

Anhang B – Datenbank (PostgreSQL)

B1 Tabellenübersicht



pgAdmin 4

File Object Tools Edit View Window Help

Welcome | lernapp/postgres@PostgreSQL 16*

lernapp/postgres@PostgreSQL 16

No limit

Query | Query History | Scratch Pad

```
1  CREATE TABLE IF NOT EXISTS benutzer (
2    benutzer_id BIGSERIAL PRIMARY KEY,
3    email       VARCHAR(255) UNIQUE NOT NULL,
4    passwort    VARCHAR(255) NOT NULL,
5    name        VARCHAR(255) NOT NULL
6  );
7
8  CREATE TABLE IF NOT EXISTS thema (
9    thema_id BIGSERIAL PRIMARY KEY,
10   titel    VARCHAR(255) NOT NULL
11 );
12
13 CREATE TABLE IF NOT EXISTS frage (
14   frage_id   BIGSERIAL PRIMARY KEY,
15   frage_text TEXT NOT NULL,
16   thema_id   BIGINT NOT NULL REFERENCES thema(thema_id)
17 );
18
19 CREATE TABLE IF NOT EXISTS antwort (
20   antwort_id   BIGSERIAL PRIMARY KEY,
```

Data Output | Messages | Notifications

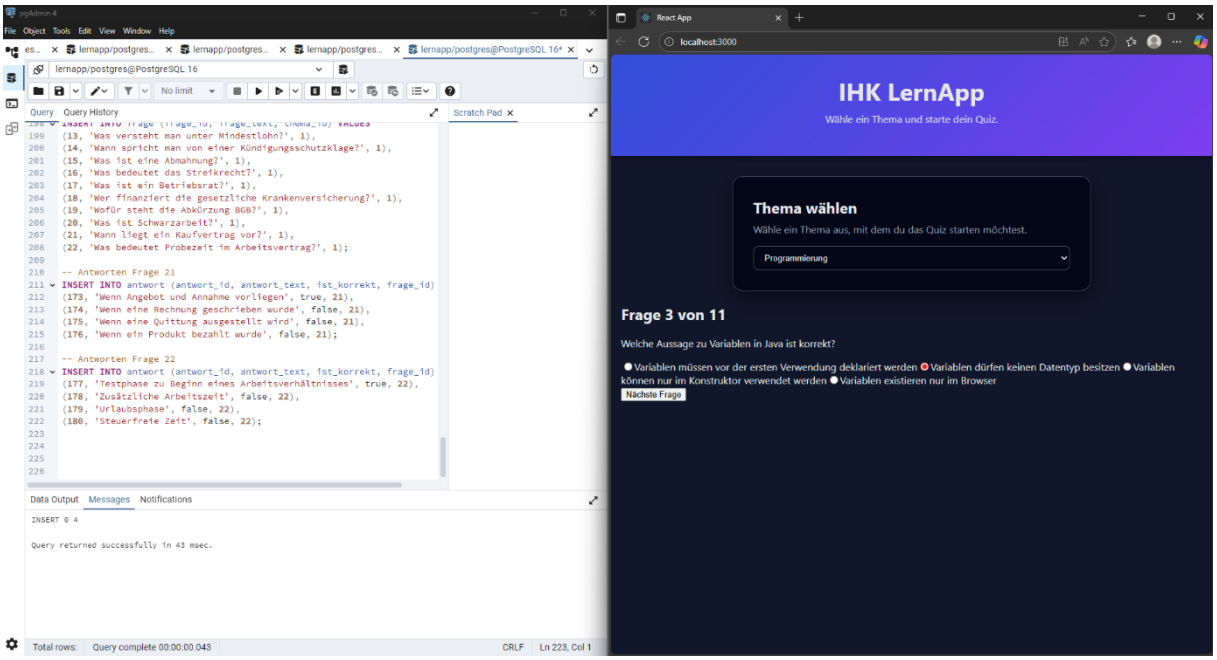
```
CREATE TABLE

Query returned successfully in 653 msec.
```

Total rows: | Query complete 00:00:00.660 | CRLF | Ln 33, Col 1

B2 Tabellenansichten (Frage, Antwort, Benutzer Ergebnis

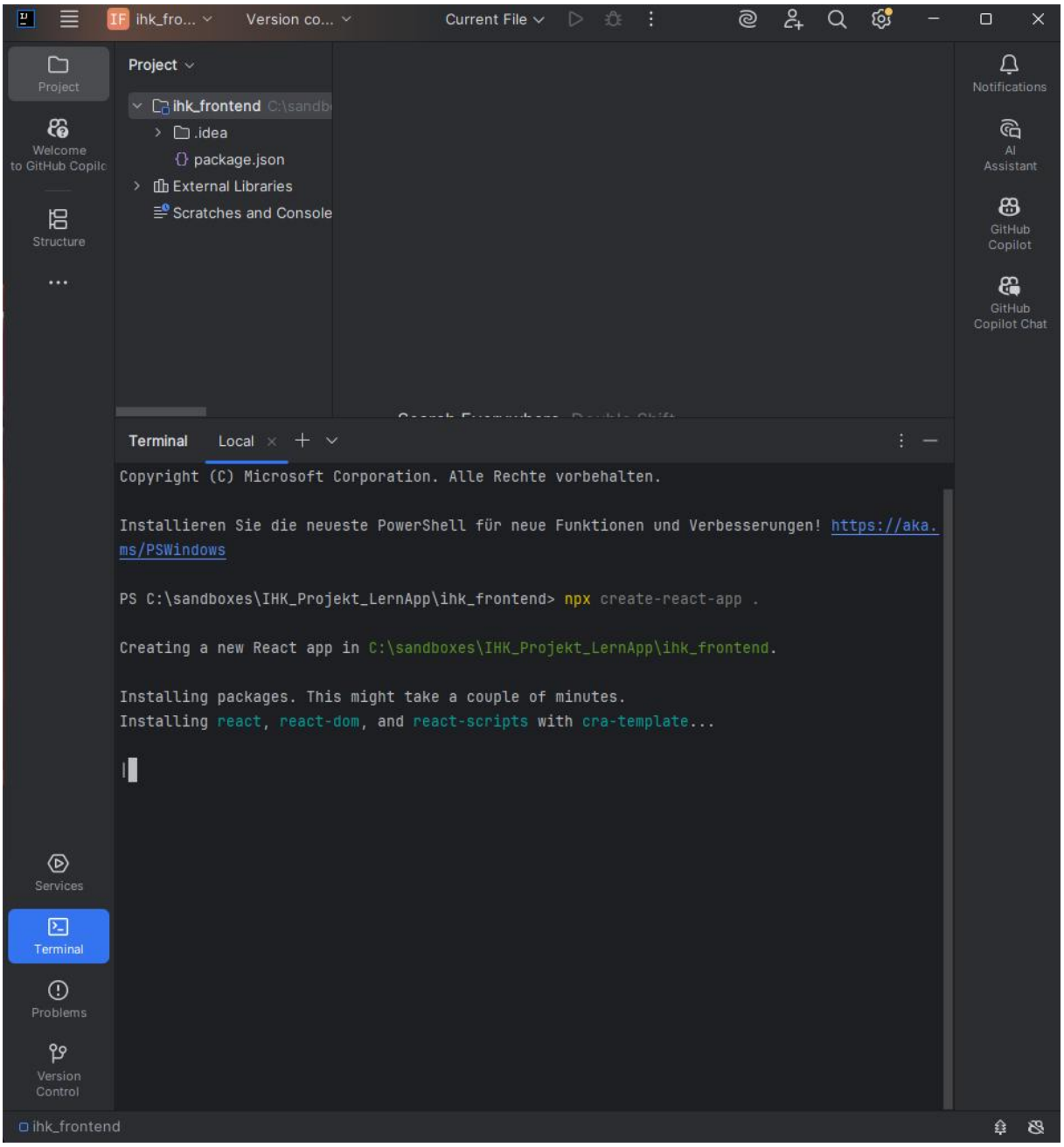
```
-- Antworten Frage 28
-------------------------------------------------------
INSERT INTO antwort (antwort_id, antwort_text, ist_korrekt, frage_id)
(201, 'Wirtschaftliche Leistung eines Landes pro Jahr', true, 28),
(202, 'Importierte Warenmenge', false, 28),
(203, 'Einkommen pro Haushalt', false, 28),
(204, 'Anzahl der Firmen im Land', false, 28);

-------------------------------------------------------
-- Antworten Frage 29
-------------------------------------------------------
INSERT INTO antwort (antwort_id, antwort_text, ist_korrekt, frage_id)
(205, 'Die Bundesregierung', true, 29),
(206, 'Das Bundesverfassungsgericht', false, 29),
(207, 'Die Europäische Union', false, 29),
(208, 'Gewerkschaften', false, 29);

-------------------------------------------------------
-- Antworten Frage 30
-------------------------------------------------------
INSERT INTO antwort (antwort_id, antwort_text, ist_korrekt, frage_id)
(209, 'Schonender Umgang mit Ressourcen', true, 30),
(210, 'Steuern auf Energieverbrauch senken', false, 30),
(211, 'Nur wirtschaftliches Wachstum fördern', false, 30),
(212, 'Nur Mülltrennung', false, 30);
INSERT INTO frage (frage_id, frage_text, thema_id) VALUES
(30, 'Was versteht man unter „ökologischer Nachhaltigkeit“?', 1)
ON CONFLICT (frage_id) DO NOTHING;

INSERT INTO antwort (antwort_id, antwort_text, ist_korrekt, frage_id)
(209, 'Schonender Umgang mit Ressourcen', true, 30),
(210, 'Steuern auf Energieverbrauch senken', false, 30),
(211, 'Nur wirtschaftliches Wachstum fördern', false, 30),
(212, 'Nur Mülltrennung', false, 30);
```

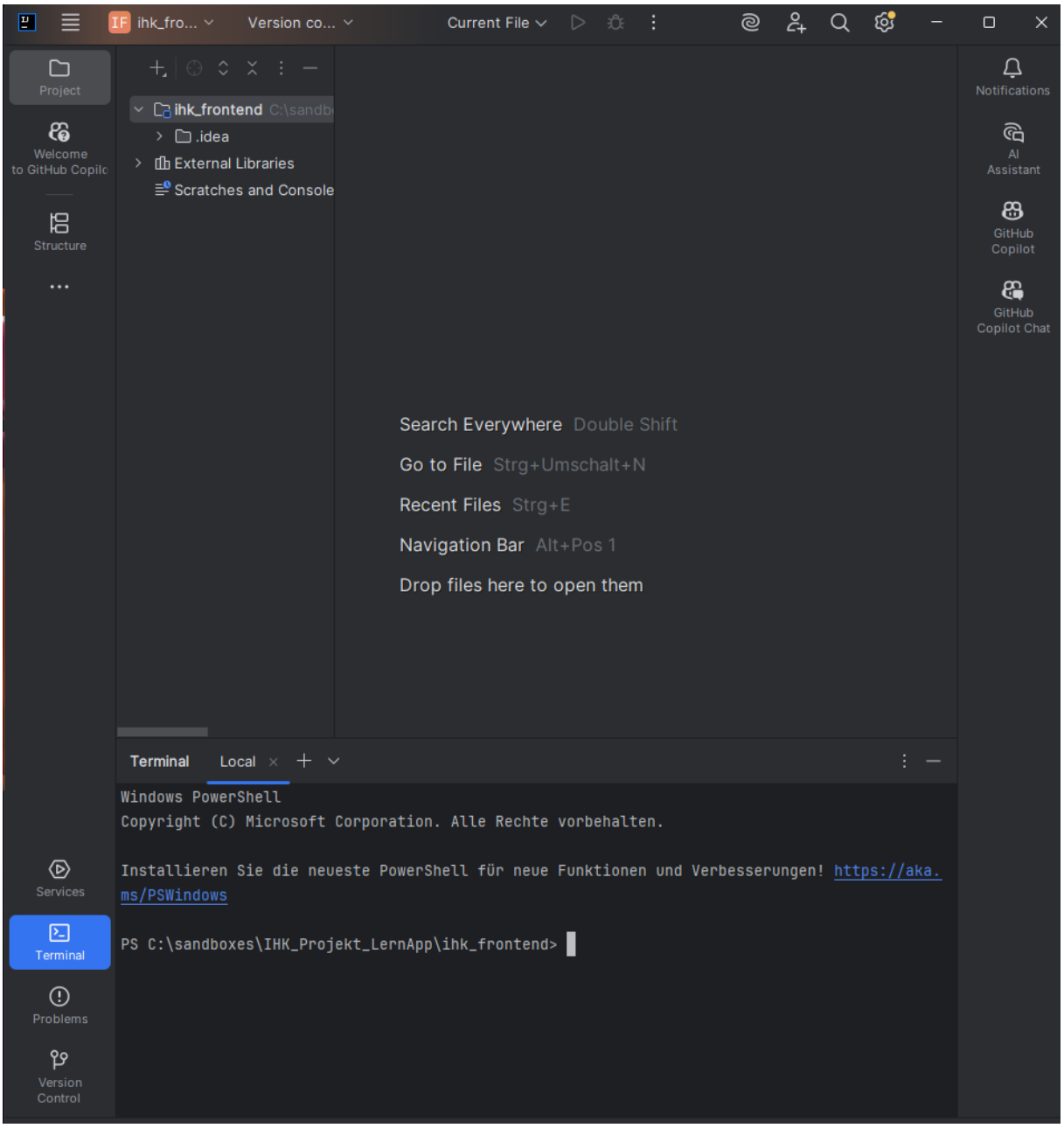
Total rows: Query complete 00:00:00.122 CRLF Ln 318, Col 1

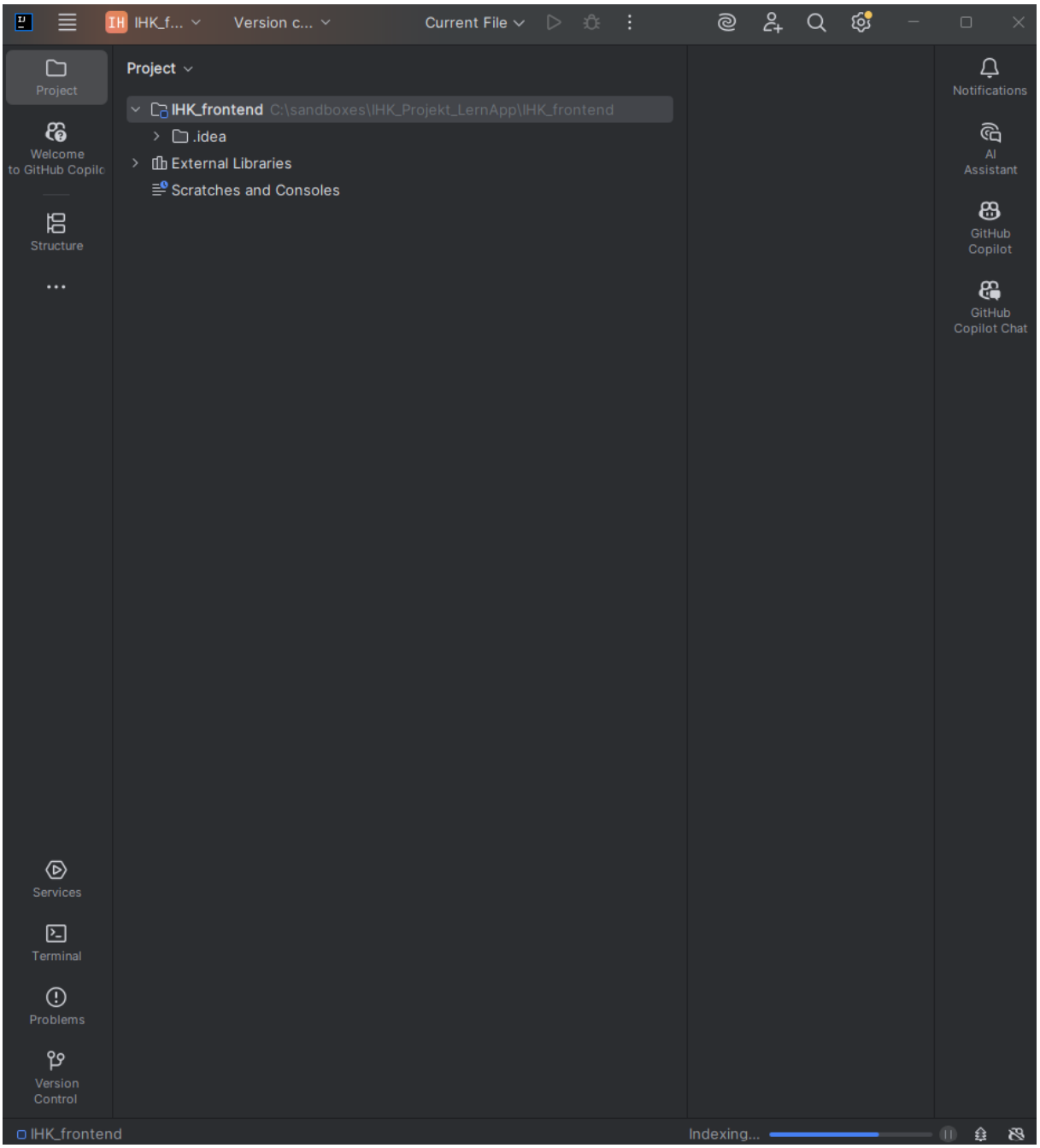

pgAdmin 4
File Object Tools Edit View Window Help
lernapp/postgres@PostgreSQL 16
No limit
Query Query History
Scratch Pad
(13, 'Was versteht man unter Mindestlohn?', 1),
(14, 'Wann spricht man von einer Kündigungsschutzklage?', 1),
(15, 'Was ist eine Abmahnung?', 1),
(16, 'Was bedeutet das Streikrecht?', 1),
(17, 'Was ist ein Betriebsrat?', 1),
(18, 'Wer finanziert die gesetzliche Krankenversicherung?', 1),
(19, 'Wofür steht die Abkürzung BGB?', 1),
(20, 'Was ist Schwarzarbeit?', 1),
(21, 'Wann liegt ein Kaufvertrag vor?', 1),
(22, 'Was bedeutet Probezeit im Arbeitsvertrag?', 1);
-- Antworten Frage 21
INSERT INTO antwort (antwort_id, antwort_text, ist_korrekt, frage_id)
(173, 'Wenn Angebot und Annahme vorliegen', true, 21),
(174, 'Wenn eine Rechnung geschrieben wurde', false, 21),
(175, 'Wenn eine Quittung ausgestellt wird', false, 21),
(176, 'Wenn ein Produkt bezahlt wurde', false, 21);
-- Antworten Frage 22
INSERT INTO antwort (antwort_id, antwort_text, ist_korrekt, frage_id)
(177, 'Testphase zu Beginn eines Arbeitsverhältnisses', true, 22),
(178, 'Zusätzliche Arbeitszeit', false, 22),
(179, 'Urlaubsphase', false, 22),
(180, 'Steuerfreie Zeit', false, 22);
Data Output Messages Notifications
INSERT 0 4
Query returned successfully in 43 msec.
Total rows: Query complete 00:00:00.043
CRLF Ln 223, Col 1
React App
localhost:3000
IHK LernApp
Wähle ein Thema und starte dein Quiz.
Thema wählen
Wähle ein Thema aus, mit dem du das Quiz starten möchtest.
Programmierung
Frage 3 von 11
Welche Aussage zu Variablen in Java ist korrekt?
Variablen müssen vor der ersten Verwendung deklariert werden
Variablen dürfen keinen Datentyp besitzen
Variablen können nur im Konstruktor verwendet werden
Variablen existieren nur im Browser
Nächste Frage

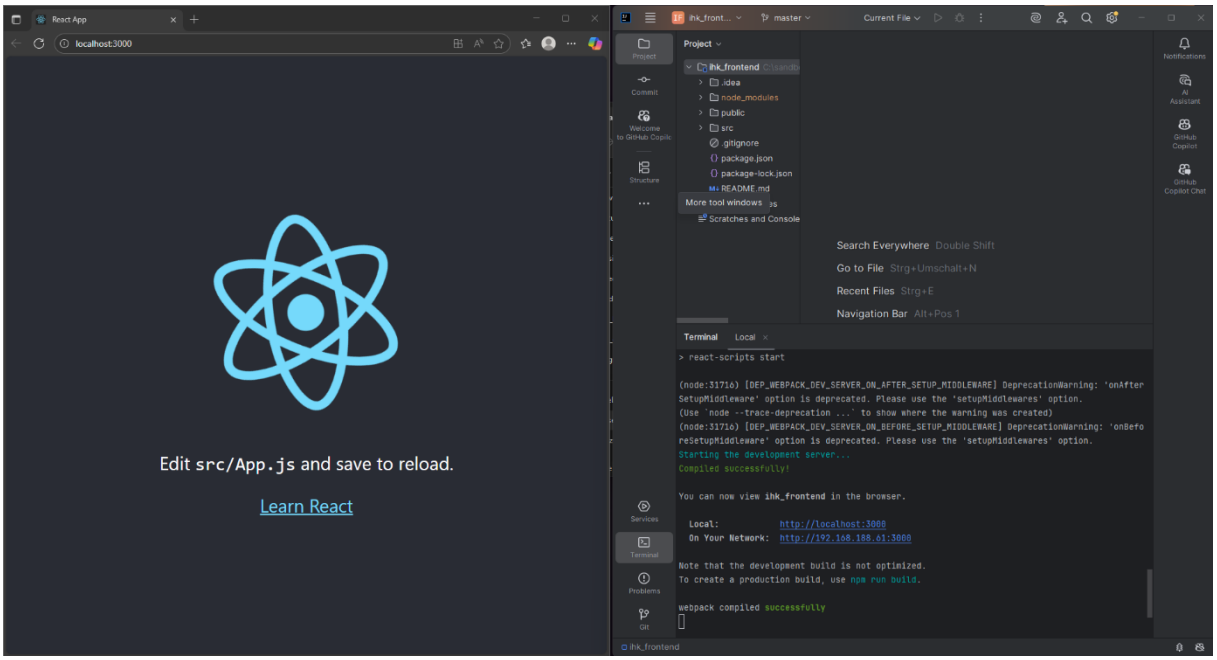
Anhang C – Frontend (React)

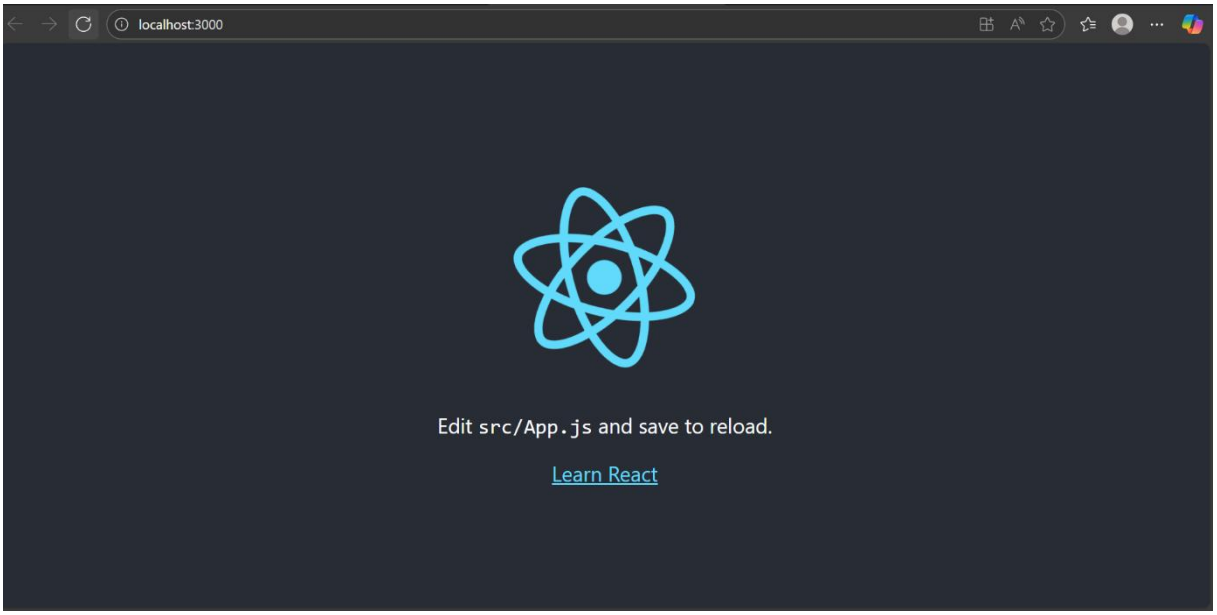
C1 Initialer Start des Frontends

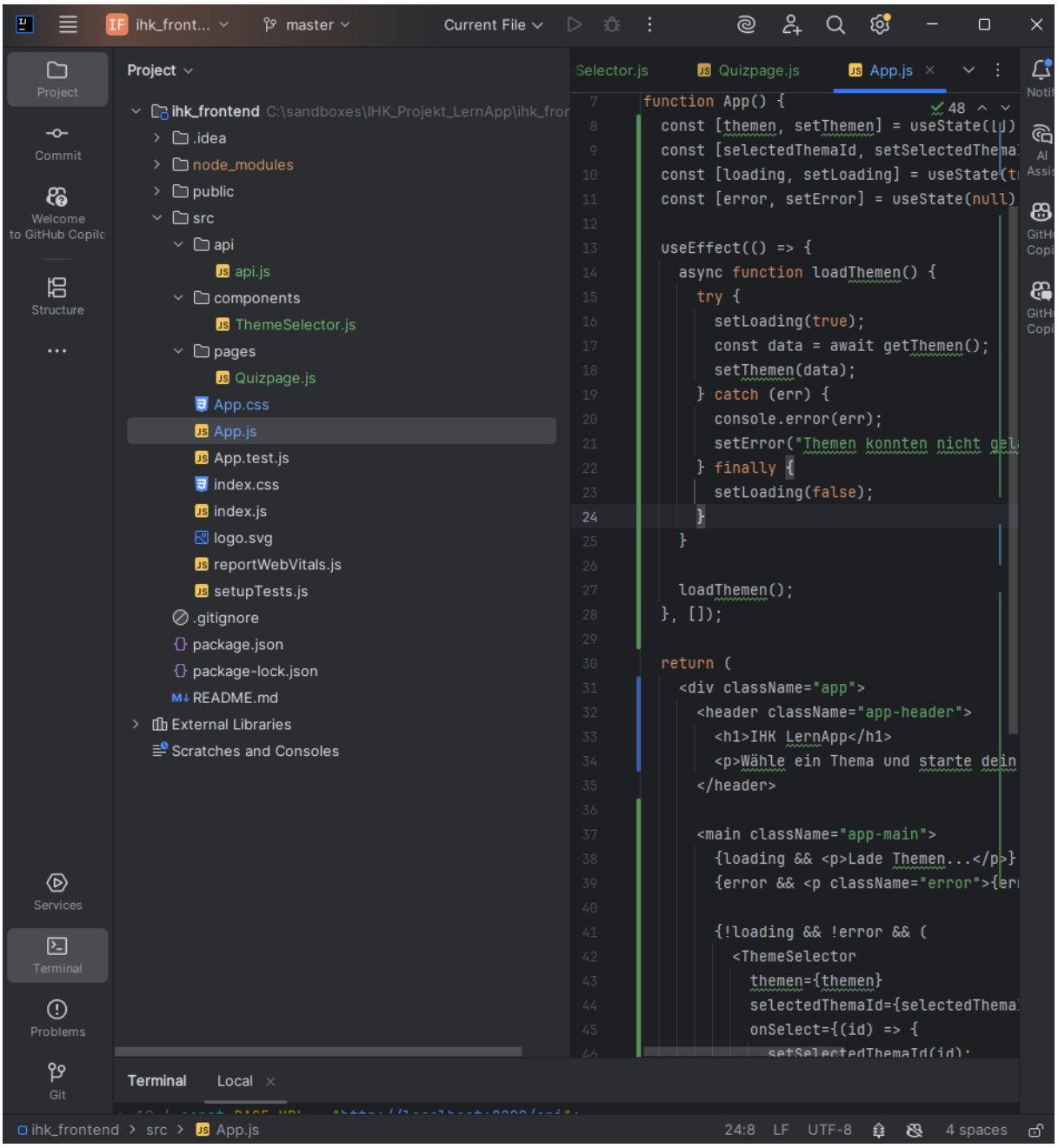



ihk_fro...
Version co...
Current File
Project
Welcome to GitHub Copilo
Structure
ihk_frontend C:\sandb
.idea
External Libraries
Scratches and Console
Notifications
AI Assistant
GitHub Copilot
GitHub Copilot Chat
Search Everywhere Double Shift
Go to File Strg+Umschalt+N
Recent Files Strg+E
Navigation Bar Alt+Pos 1
Drop files here to open them
Terminal
Local
Windows PowerShell
Copyright (C) Microsoft Corporation. Alle Rechte vorbehalten.
Installieren Sie die neueste PowerShell für neue Funktionen und Verbesserungen! https://aka.ms/PSWindows
PS C:\sandboxes\IHK_Projekt_LernApp\ihk_frontend>
Services
Terminal
Problems
Version Control


IHK_f...
Version c...
Current File
Project
Welcome to GitHub Copilc
Structure
Services
Terminal
Problems
Version Control
Project
IHK_frontend C:\sandboxes\IHK_Projekt_LernApp\IHK_frontend
.idea
External Libraries
Scratches and Consoles
Notifications
AI Assistant
GitHub Copilot
GitHub Copilot Chat
IHK_frontend
Indexing...

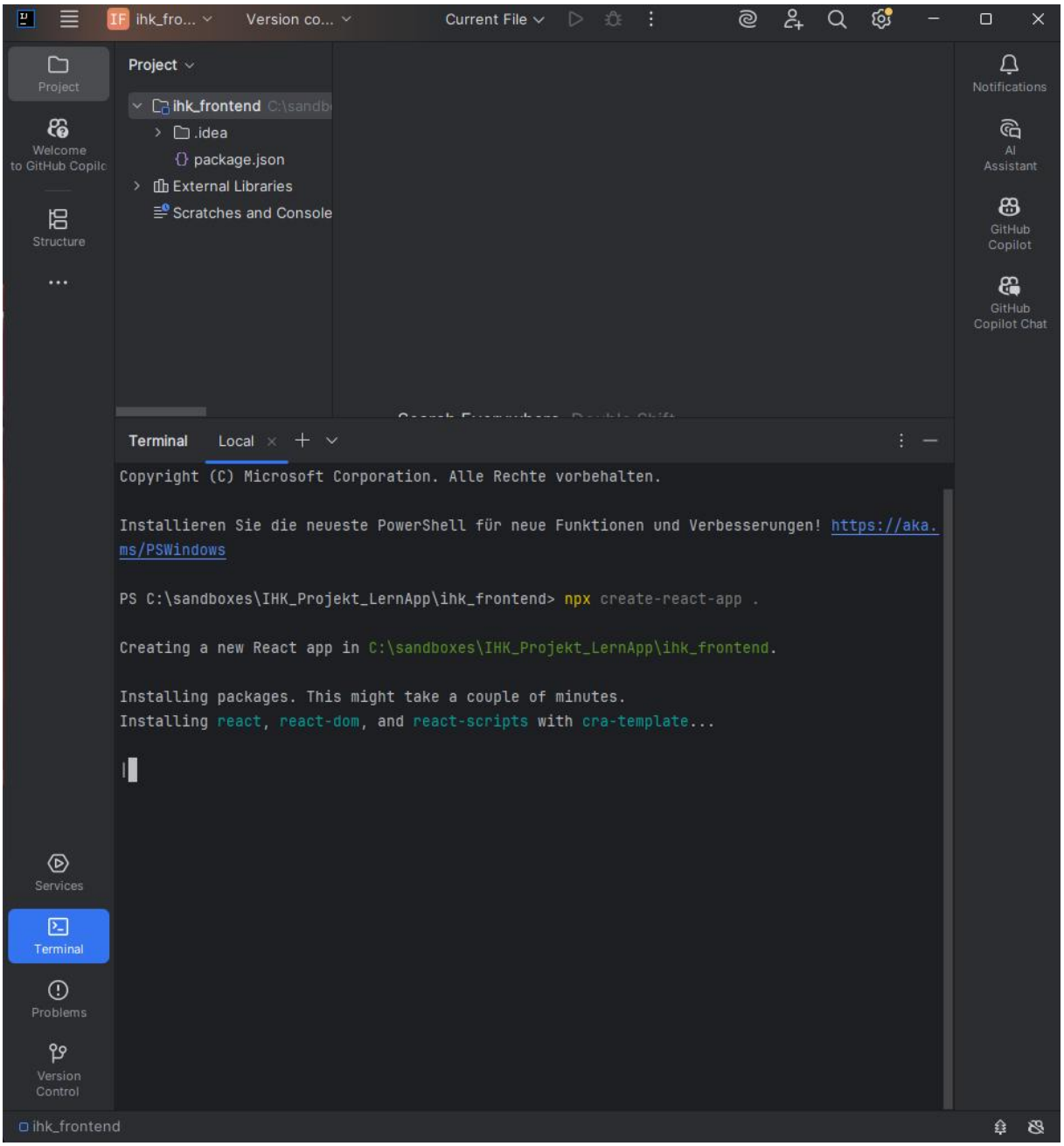

localhost:3000
Edit src/App.js and save to reload.
Learn React
Project
ihk_frontend
.idea
node_modules
public
src
.gitignore
package.json
package-lock.json
README.md
Scratches and Console
Search Everywhere Double Shift
Go to File Strg+Umschalt+N
Recent Files Strg+E
Navigation Bar Alt+Pos 1
Terminal
> react-scripts start
(node:31716) [DEP_WEBPACK_DEV_SERVER_ON_AFTER_SETUP_MIDDLEWARE] DeprecationWarning: 'onAfter SetupMiddleware' option is deprecated. Please use the 'setupMiddlewares' option.
(Use `node --trace-deprecation ...` to show where the warning was created)
(node:31716) [DEP_WEBPACK_DEV_SERVER_ON_BEFORE_SETUP_MIDDLEWARE] DeprecationWarning: 'onBefo reSetupMiddleware' option is deprecated. Please use the 'setupMiddlewares' option.
Starting the development server...
Compiled successfully!
You can now view ihk_frontend in the browser.
Local: http://localhost:3000
On Your Network: http://192.168.188.61:3000
Note that the development build is not optimized.
To create a production build, use npm run build.
webpack compiled successfully


localhost:3000
Edit src/App.js and save to reload.
Learn React


ihk_front...
master
Current File
Project
ihk_frontend C:\sandboxes\IHK_Projekt_LernApp\ihk_fror
.idea
node_modules
public
src
api
api.js
components
ThemeSelector.js
pages
Quizpage.js
App.css
App.js
App.test.js
index.css
index.js
logo.svg
reportWebVitals.js
setupTests.js
.gitignore
package.json
package-lock.json
README.md
External Libraries
Scratches and Consoles
Selector.js
Quizpage.js
App.js
function App() {
const [themen, setThemen] = useState([])
const [selectedThemaId, setSelectedThema
const [loading, setLoading] = useState(tr
const [error, setError] = useState(null)
useEffect(() => {
async function loadThemen() {
try {
setLoading(true);
const data = await getThemen();
setThemen(data);
} catch (err) {
console.error(err);
setError("Themen konnten nicht gel
} finally {
setLoading(false);
}
}
loadThemen();
}, []);
return (

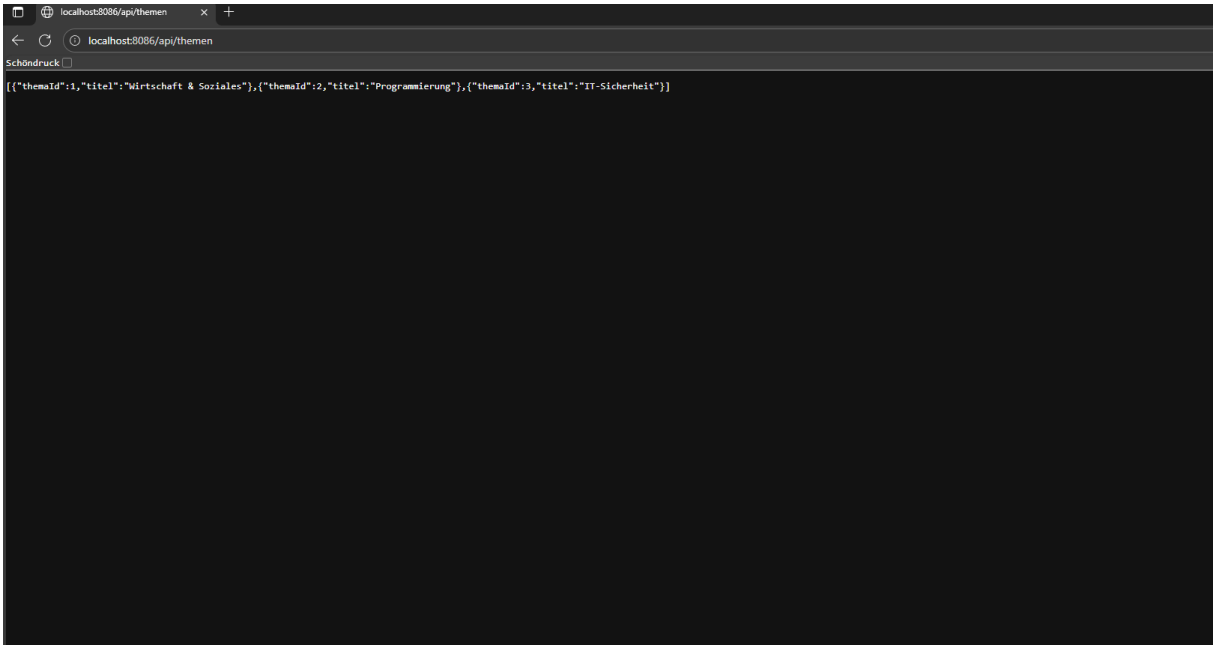
<header className="app-header">
<h1>IHK LernApp</h1>
Wähle ein Thema und starte dein
</header>
<main className="app-main">
{loading && Lade Themen... }
{error && {er
{!loading && !error && (
<ThemeSelector
themen={themen}
selectedThemaId={selectedThema
onSelect={(id) => {
Terminal
Local
ihk_frontend > src > App.js
24:8
LF
UTF-8
4 spaces

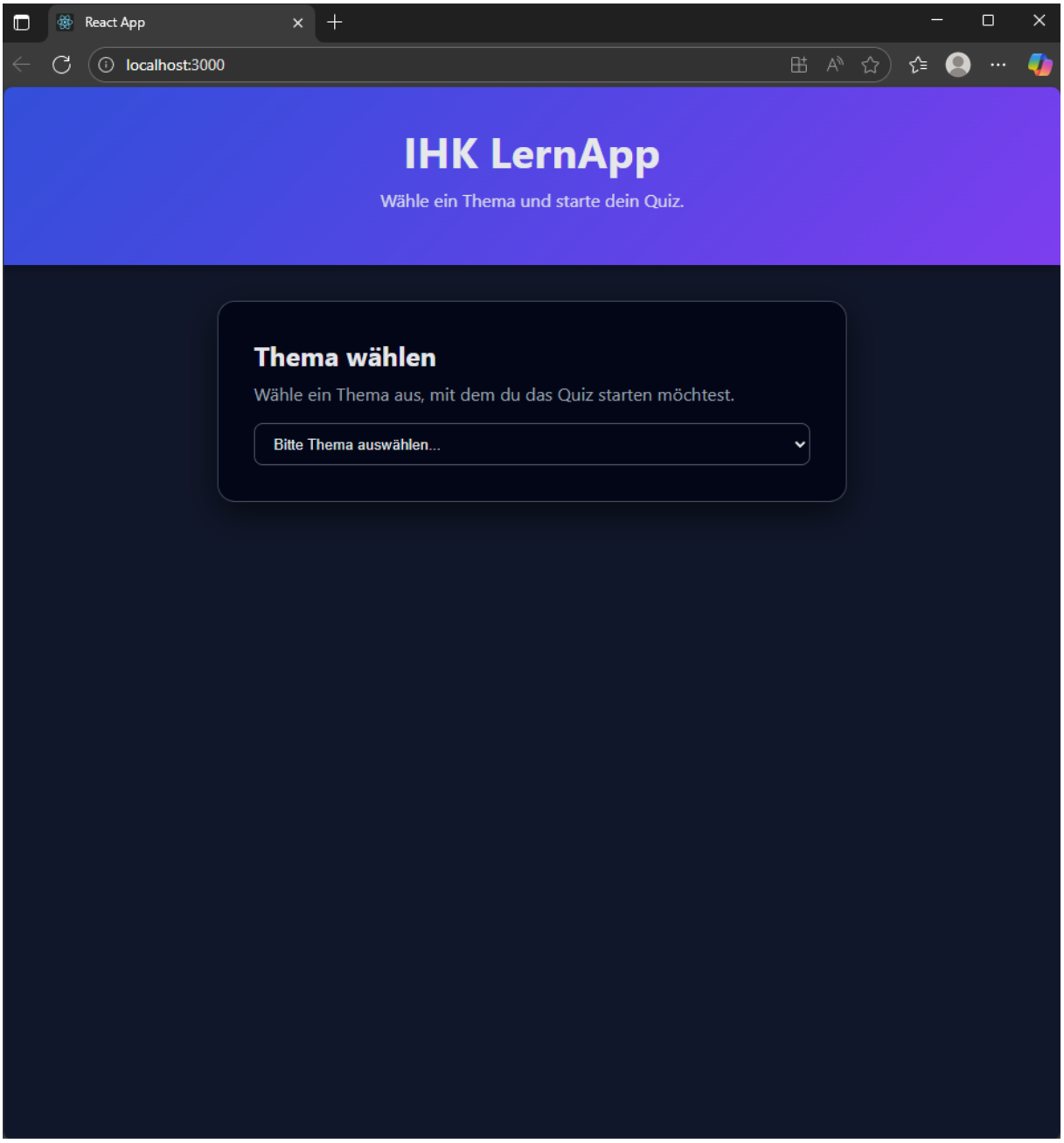
C2 npm / npx

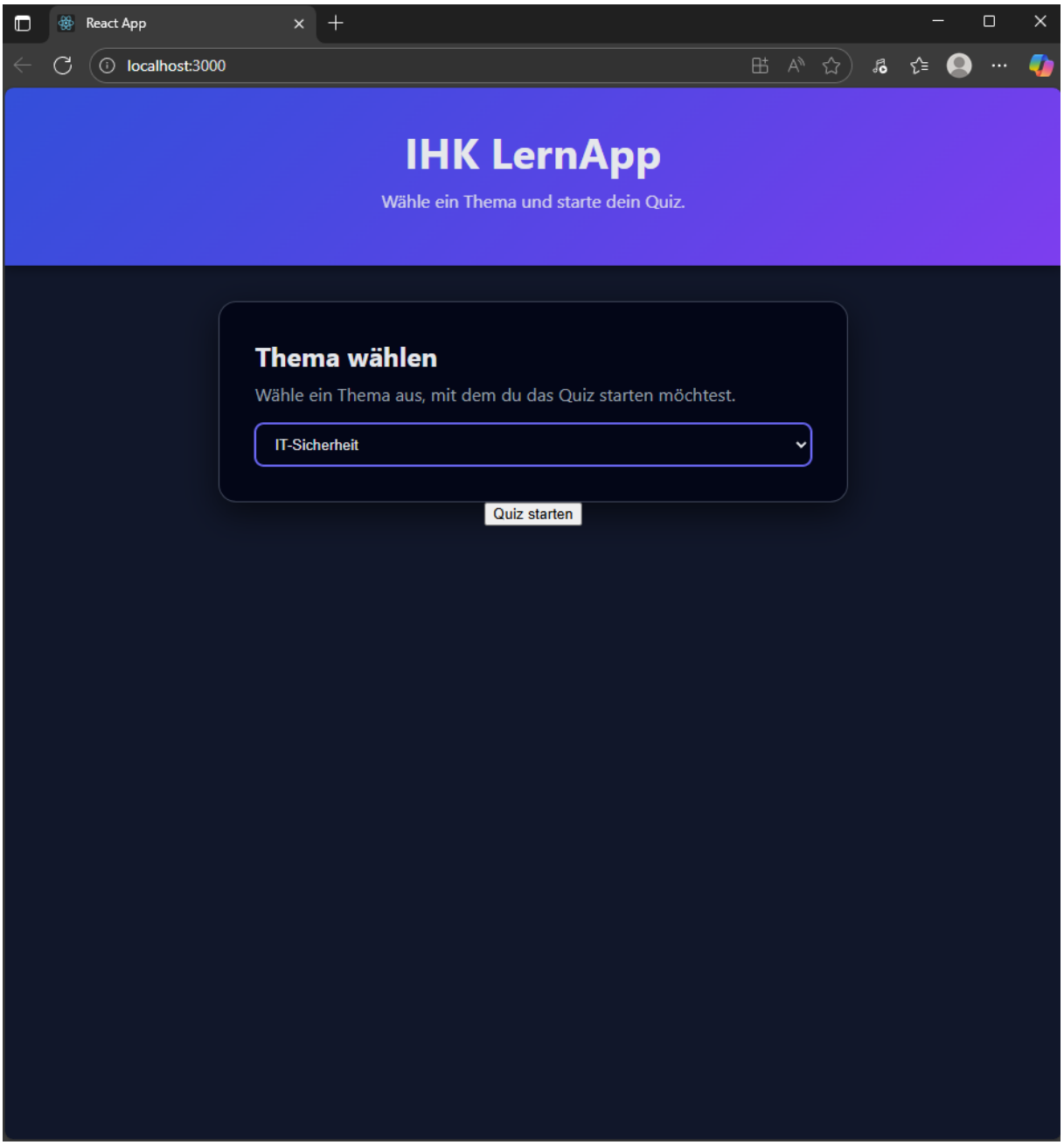

ihk_fro...
Version co...
Current File
Project
ihk_frontend
.idea
package.json
External Libraries
Scratches and Console
Notifications
AI Assistant
GitHub Copilot
GitHub Copilot Chat
Welcome to GitHub Copilc
Structure
Services
Terminal
Problems
Version Control
Terminal
Local
Copyright (C) Microsoft Corporation. Alle Rechte vorbehalten.
Installieren Sie die neueste PowerShell für neue Funktionen und Verbesserungen! https://aka.ms/PSWindows
PS C:\sandboxes\IHK_Projekt_LernApp\ihk_frontend> npx create-react-app .
Creating a new React app in C:\sandboxes\IHK_Projekt_LernApp\ihk_frontend.
Installing packages. This might take a couple of minutes.
Installing react, react-dom, and react-scripts with cra-template...
ihk_frontend

Anhang D – Funktionstests

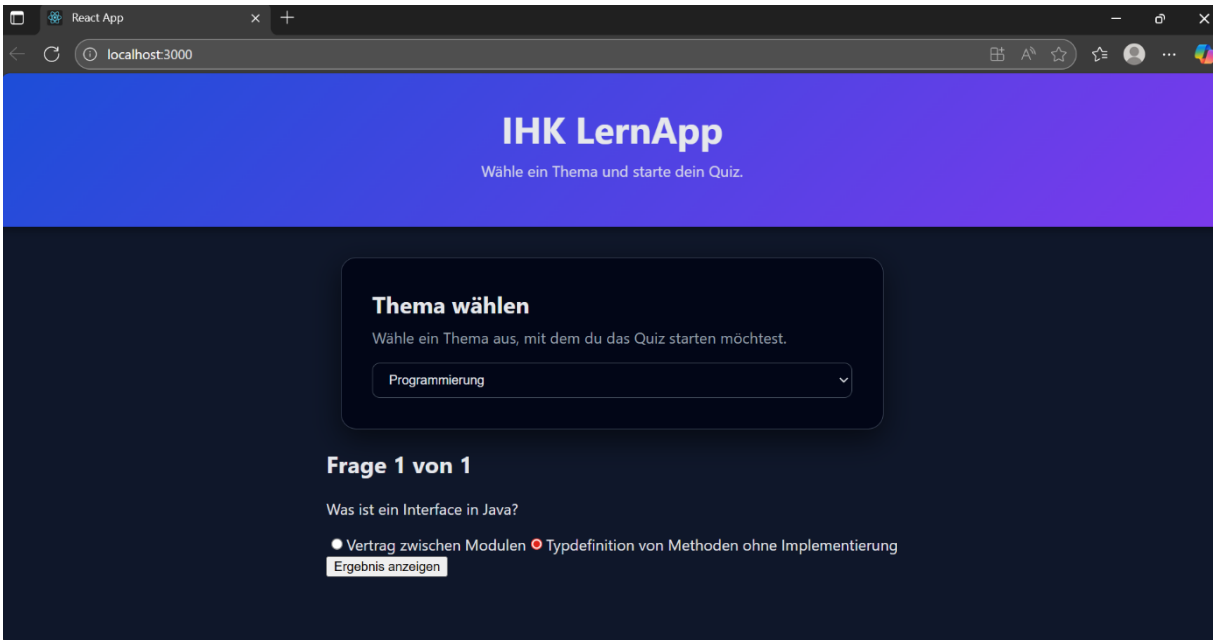
D1 Test: Themenauswahl

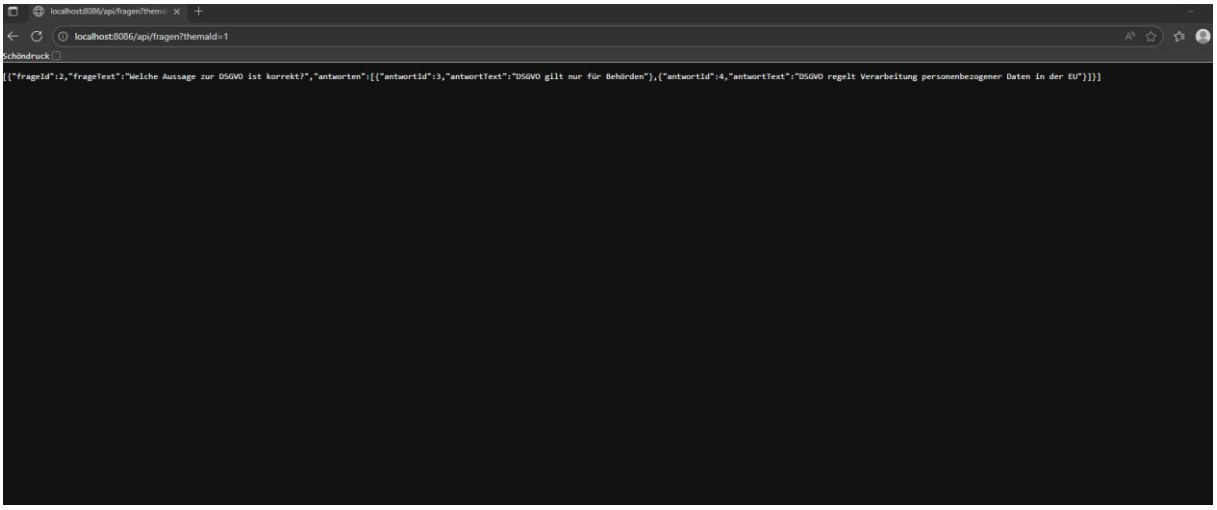



React App
localhost:3000
IHK LernApp
Wähle ein Thema und starte dein Quiz.
Thema wählen
Wähle ein Thema aus, mit dem du das Quiz starten möchtest.
Bitte Thema auswählen...

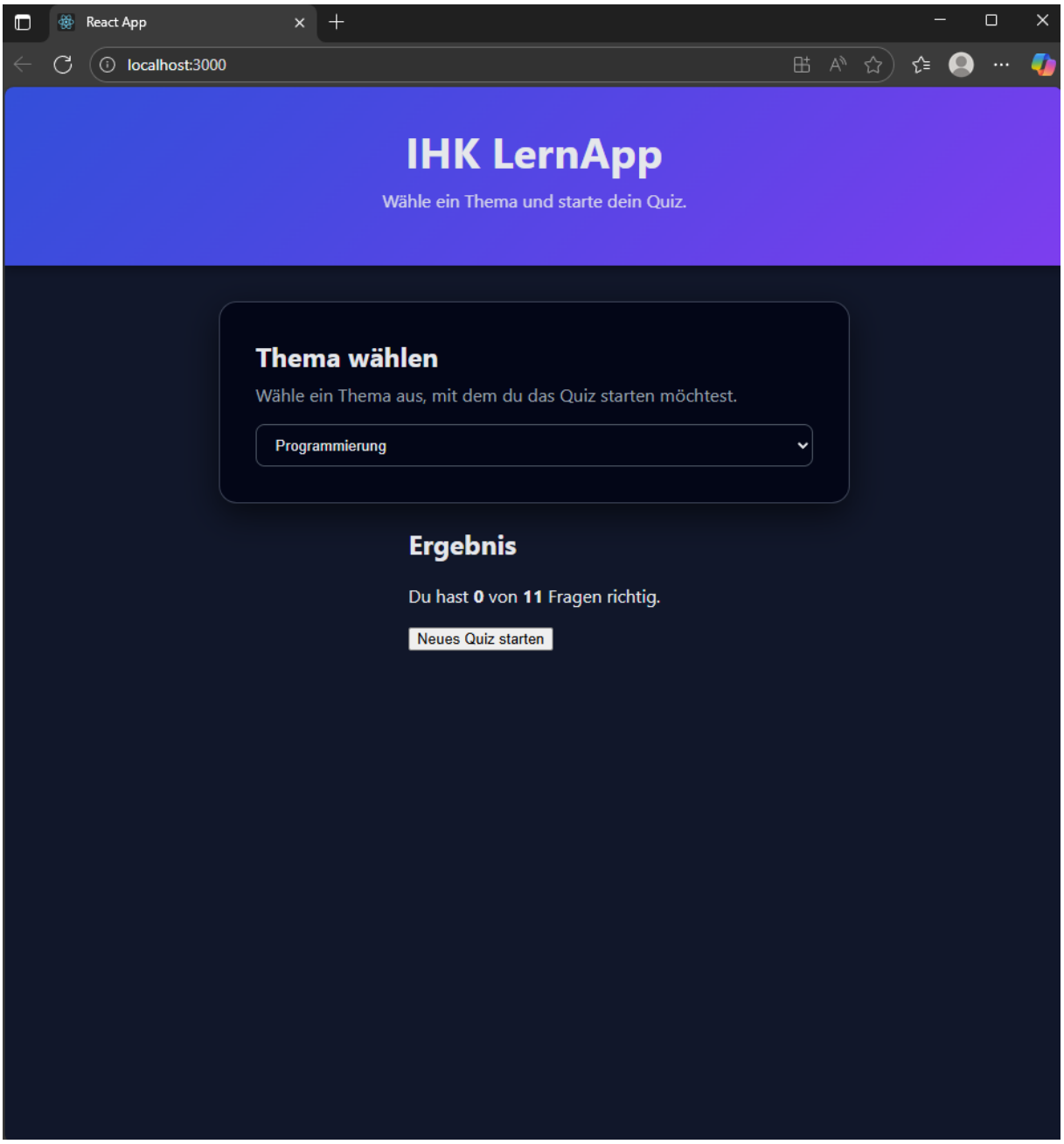

React App
localhost:3000
IHK LernApp
Wähle ein Thema und starte dein Quiz.
Thema wählen
Wähle ein Thema aus, mit dem du das Quiz starten möchtest.
IT-Sicherheit
Quiz starten

D2 Test: Fragen laden

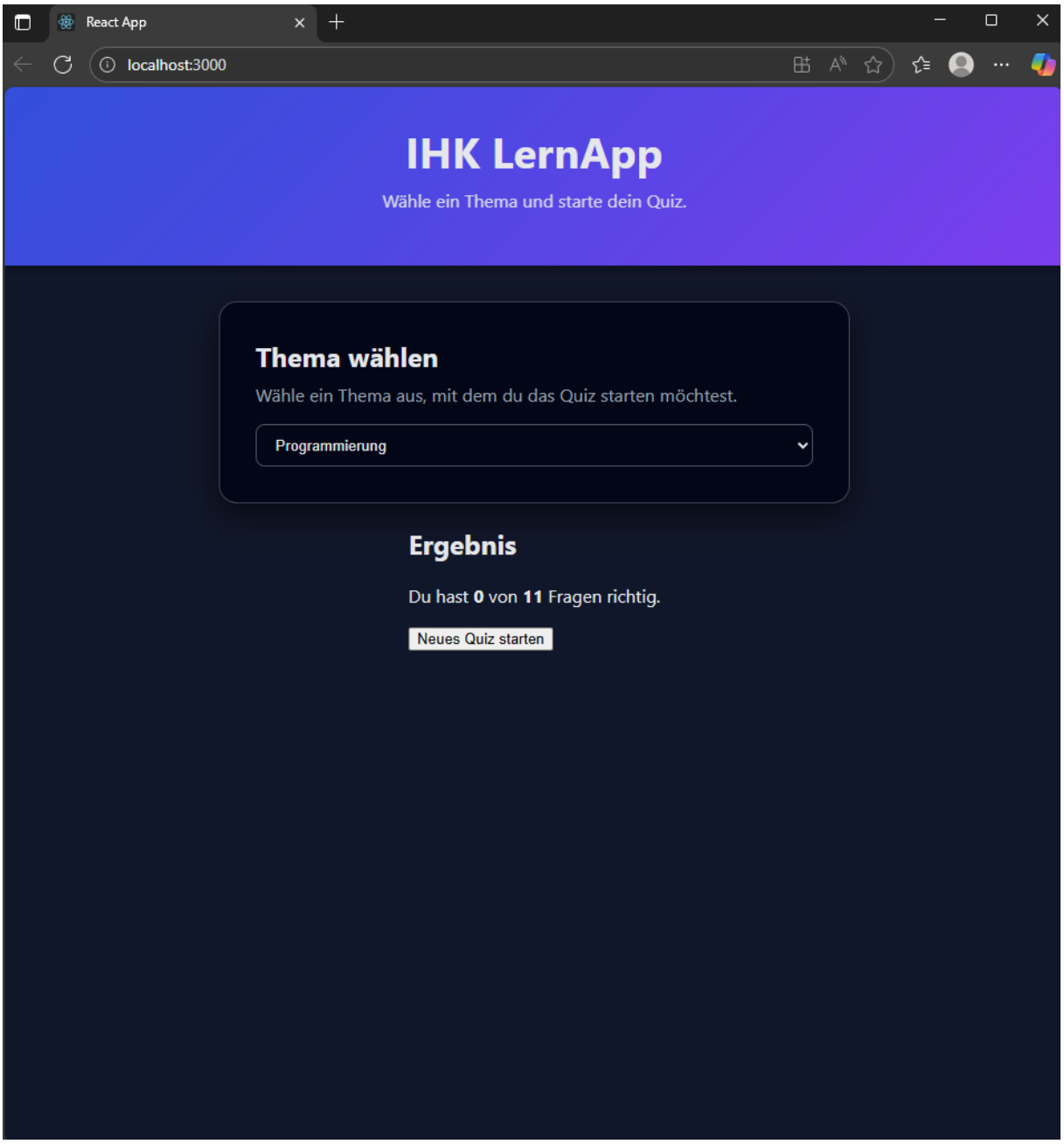




D3 Test: Antworten & Ergebnisanzeige



D5 Test: Quiz-Abschluss & Neustart



Quellenverzeichnis

- Spring Boot Dokumentation – https://spring.io/projects/spring-boot
- React Dokumentation – https://react.dev
- PostgreSQL Dokumentation – https://www.postgresql.org/docs
- StackOverflow (Recherche) – https://stackoverflow.com
- Mozilla Developer Network (JavaScript/React) – https://developer.mozilla.org

- Eigene Umsetzung (Quellcode Backend, Frontend, Datenbank)